

Coasterbot

Testkonzept



Dennis Borutta, B. Eng	dennis.borutta@outlook.com
Stefan Etringer, B. Eng	mail@stefan.works
Gereon Such, M. Sc.	gereonsuch@gmail.com

Versionshistorie

Version	Datum	Bearbeiter	Änderungen
1	19.08.2026	Etringer	Erstfassung der Alternativenbeschreibung

Inhaltsverzeichnis

1	Einleitung	11
1.1	Zweck der Testspezifikation	11
1.2	Zielsetzung der Testdurchführung	11
1.3	Geltungsbereich	11
1.4	Testobjekt	12
1.4.1	Hardware	12
1.4.2	Software	12
1.4.3	Simulation	13
1.5	Testgrundlagen	13
1.6	Teststrategie	13
1.7	Komponententests	13
1.8	Integrationstests	14
1.9	Simulationstests	14
1.10	Systemtests	14
1.11	Testumgebung	14
1.11.1	Entwicklungsumgebung	15
1.11.2	Simulationsumgebung	15
1.11.3	Physischer Prototyp	15
1.12	Bewertung von Testergebnissen	15
1.13	Aufbau der Testspezifikation	16
1.14	Zusammenfassung	16
2	Testmethodik und Testmanagement	17
2.1	Ziel der Testmethodik	17
2.2	Testvorgehensmodell	17
2.2.1	Testplanung	17
2.2.2	Testentwurf	18
2.3	Testebenen	18
2.3.1	Komponententest	18
2.3.2	Integrationstest	19
2.3.3	Simulationstest	19
2.3.4	Systemtest	20
2.4	Testarten	20
2.4.1	Funktionstest	20
2.4.2	Sicherheitstest	20
2.4.3	Belastungstest	21
2.4.4	Regressionstest	21
2.5	Priorisierung der Testfälle	21
2.5.1	Priorität P1 – Kritische Tests	22
2.5.2	Priorität P2 – Kernfunktionen	22
2.5.3	Priorität P3 – Erweiterungen	22

2.6	Testdaten und Testszenarien	22
2.6.1	Normalszenario	23
2.6.2	Hindernisszenario	23
2.6.3	Grenzszenario	23
2.6.4	Fehlerszenario	23
2.7	Testautomatisierung	24
2.8	Fehlerverwaltung	24
2.9	Testabschluss	25
2.10	Zusammenfassung	25
3	Komponententests	25
3.1	Ziel der Komponententests	25
3.2	Testumgebung für Komponententests	26
3.3	Testfallstruktur	26
3.4	Komponententest: Hardwareabstraktion	27
3.4.1	Testfall CT-HW-001: Verarbeitung von Sensordaten	27
3.5	Komponententest: Lokalisierung	28
3.5.1	Testfall CT-LOC-001: Positionsaktualisierung	28
3.6	Komponententest: Navigation	29
3.6.1	Testfall CT-NAV-001: Berechnung einer gültigen Route	29
3.6.2	Testfall CT-NAV-002: Hinderniserkennung	30
3.7	Komponententest: Bewegungssteuerung	30
3.7.1	Testfall CT-MOT-001: Umsetzung von Bewegungsbefehlen	30
3.8	Komponententest: Untersetzerhandling	31
3.8.1	Testfall CT-CST-001: Aufnahme eines Getränkeuntersetzers	31
3.8.2	Testfall CT-CST-002: Fehler bei Untersetzeraufnahme	32
3.9	Komponententest: Sicherheitskomponente	33
3.9.1	Testfall CT-SAF-001: Sicherheitsstopp	33
3.10	Komponententest: Systemüberwachung	34
3.10.1	Testfall CT-MON-001: Fehlererkennung und Protokollierung	34
3.11	Zusammenfassung der Komponententests	35
4	Integrationstests	35
4.1	Ziel der Integrationstests	35
4.2	Testumgebung für Integrationstests	36
4.3	Testfallstruktur	36
4.4	Integrationstest: Sensorik und Navigation	36
4.4.1	Testfall IT-SEN-NAV-001: Verarbeitung von Hindernisdaten	36
4.5	Integrationstest: Lokalisierung und Navigation	38
4.5.1	Testfall IT-LOC-NAV-001: Zielnavigation mit Positionsdaten	38
4.6	Integrationstest: Navigation und Bewegungssteuerung	39
4.6.1	Testfall IT-NAV-MOT-001: Umsetzung von Navigationsbefehlen	39
4.7	Integrationstest: Sicherheitslogik und Bewegungssteuerung	40
4.7.1	Testfall IT-SAF-MOT-001: Unterbrechen einer Bewegung bei Gefahr	40

4.8	Integrationstest: Untersetzerhandling und Navigation	41
4.8.1	Testfall IT-NAV-CST-001: Navigation zu einem Untersetzer und Aufnahme	41
4.9	Integrationstest: Systemüberwachung und Komponenten	42
4.9.1	Testfall IT-MON-001: Fehlerweitergabe im Gesamtsystem	42
4.10	Integrationstest: Hardwareabstraktion und Simulation	43
4.10.1	Testfall IT-HW-SIM-001: Austausch realer Hardware durch Simulation	43
4.11	Zusammenfassung der Integrationstests	44
5	Simulationstests	45
5.1	Ziel der Simulationstests	45
5.2	Testumgebung der Simulation	45
5.3	Anforderungen an Simulationstests	46
5.4	Simulationstest: Grundlegende Navigation	46
5.4.1	Testfall ST-SIM-001: Navigation zu einem Zielpunkt	46
5.5	Simulationstest: Hinderniserkennung und Ausweichverhalten	48
5.5.1	Testfall ST-SIM-002: Dynamische Hindernisvermeidung	48
5.6	Simulationstest: Tischkantenerkennung	49
5.6.1	Testfall ST-SIM-003: Verhindern des Verlassens der Tischfläche	49
5.7	Simulationstest: Reproduzierbarkeit von Fahrmanövern	50
5.7.1	Testfall ST-SIM-004: Wiederholbarkeit identischer Bewegungsabläufe	50
5.8	Simulationstest: Sensorausfall	51
5.8.1	Testfall ST-SIM-005: Reaktion auf fehlerhafte Sensordaten	51
5.9	Simulationstest: Aktorfehler	52
5.9.1	Testfall ST-SIM-006: Verhalten bei fehlerhafter Motorsteuerung	52
5.10	Simulationstest: Untersetzerhandling	52
5.10.1	Testfall ST-SIM-007: Aufnahme und Platzierung eines Untersetzers	52
5.11	Simulationstest: Validierung Simulation gegen erwartetes Verhalten	53
5.11.1	Testfall ST-SIM-008: Vergleich Simulation und Systemverhalten	53
5.12	Zusammenfassung der Simulationstests	54
6	Systemtests	55
6.1	Ziel der Systemtests	55
6.2	Testumgebung	55
6.3	Systemteststruktur	56
6.4	Systemtest: Inbetriebnahme	56
6.4.1	Testfall SYS-001: Systemstart und Initialisierung	56
6.5	Systemtest: Navigation auf Tischfläche	57
6.5.1	Testfall SYS-002: Autonome Bewegung auf dem Tisch	57
6.6	Systemtest: Hindernisvermeidung	58
6.6.1	Testfall SYS-003: Erkennung und Umfahrung eines Hindernisses	58
6.7	Systemtest: Tischkantenerkennung	59
6.7.1	Testfall SYS-004: Schutz vor Herunterfallen vom Tisch	59

6.8	Systemtest: Untersetzeraufnahme	60
6.8.1	Testfall SYS-005: Aufnahme eines Getränkeuntersetzers	60
6.9	Systemtest: Untersetzterplatzierung	61
6.9.1	Testfall SYS-006: Platzierung eines Untersetzers	61
6.10	Systemtest: Sicherheitsstopp	62
6.10.1	Testfall SYS-007: Aktivierung des Not-Aus	62
6.11	Systemtest: Fehlerbehandlung	63
6.11.1	Testfall SYS-008: Verhalten bei Systemfehler	63
6.12	Systemtest: Energieversorgung	64
6.12.1	Testfall SYS-009: Überwachung des Akkuzustands	64
6.13	Systemtest: Gesamttablauf	64
6.13.1	Testfall SYS-010: Vollständiger Coasterbot-Arbeitsablauf	64
6.14	Zusammenfassung der Systemtests	65
7	Testergebnisse und Bewertung	66
7.1	Ziel der Testauswertung	66
7.2	Dokumentation der Testergebnisse	66
7.3	Bewertungskriterien	66
7.3.1	Bestanden	67
7.3.2	Fehlgeschlagen	67
7.3.3	Nicht ausführbar	67
7.3.4	Testabdeckung	67
7.4	Auswertung der Kernfunktionen	68
7.4.1	Navigation und Lokalisierung	68
7.4.2	Sicherheitsfunktionen	68
7.4.3	Getränkeuntersetzterhandling	69
7.4.4	Systemüberwachung	69
7.5	Fehleranalyse	69
7.5.1	Umgang mit kritischen Fehlern	70
7.5.2	Umgang mit funktionalen Fehlern	70
7.6	Bewertung der Definition of Done	70
7.6.1	Fertigungsunterlagen	71
7.6.2	Softwarearchitektur	71
7.6.3	Simulation	71
7.6.4	Testspezifikation	71
7.7	Bewertung der Projektziele	71
7.7.1	Grenzen der Bewertung	72
8	Zusammenfassung	72
8.1	Zusammenfassung und Ausblick	73
8.1.1	Zusammenfassung der Testspezifikation	73
8.1.2	Bewertung der erreichten Projektziele	73
8.1.3	Bewertung der Softwarearchitektur	74
8.1.4	Bedeutung der Simulation	74

8.1.5	Grenzen des entwickelten Prototyps	74
8.1.6	Verbesserungsmöglichkeiten	75
8.2	Fazit	76
9	Anhang	77
9.1	Anhang Vollständige Testfallmatrix (Anforderung → Test-ID → Ergebnis)	77
9.1.1	Zweck der Testfallmatrix	77
10	A.2 Übersicht der Test-ID-Struktur	77
11	A.3 Komponententests	77
12	A.4 Integrationstests	78
13	A.5 Simulationstests	79
14	A.6 Systemtests	80
15	A.7 Testabdeckung nach Anforderungsbereich	81
16	A.8 Testpriorisierung	81
16.1	Priorität P1 – Kritische Funktionen	81
16.2	Priorität P2 – Kernfunktionen	81
16.3	Priorität P3 – Optionale Funktionen	82
17	A.9 Teststatusdefinition	82
18	A.10 Zusammenfassung	82
18.0.1	Anhang B: Testprotokoll-Vorlagen	83
19	B.2 Allgemeine Testinformationen	83
19.1	Testprotokoll	83
20	B.3 Beschreibung des Testziels	83
20.1	Ziel des Tests	83
20.2	Zugeordnete Anforderungen	84
21	B.4 Testvoraussetzungen	84
22	B.5 Testaufbau	84
22.1	Beschreibung der Testumgebung	84
22.2	Verwendete Komponenten	84
23	B.6 Testdaten	85
24	B.7 Testdurchführung	85
24.1	Ablauf	85

25 B.8 Erwartetes Ergebnis	85
26 B.9 Tatsächliches Ergebnis	85
27 B.10 Bewertung des Testergebnisses	85
27.1 Testergebnis	85
27.2 Begründung	86
28 B.11 Fehlerdokumentation	86
28.1 Fehlerbeschreibung	86
28.2 Fehleranalyse	86
28.2.1 Ursache	86
28.2.2 Auswirkungen	86
28.2.3 Maßnahmen	86
29 B.12 Testabschluss	86
29.1 Zusammenfassung	86
29.2 Testergebnis bestätigt durch	87
30 B.13 Vorlage für automatisierte Tests	87
30.1 Beispiel eines automatisierten Testergebnisses	87
31 B.14 Vorlage für Simulationstests	88
31.1 Simulationsszenario	88
31.1.1 Beschreibung	88
31.1.2 Startbedingungen	88
31.1.3 Ergebnisvergleich	88
32 B.15 Vorlage für Systemtests	89
32.1 Bewertung des Systemverhaltens	89
33 B.16 Zusammenfassung	89
33.0.1 Anhang C: Traceability-Matrix Lastenheft → Pflichtenheft → Testfälle	90
34 Anhang C: Traceability-Matrix Lastenheft → Pflichtenheft → Testfälle	90
34.1 C.1 Zweck der Traceability-Matrix	90
35 C.2 Struktur der Traceability-Matrix	90
36 C.3 Navigation und Lokalisierung	90
37 C.4 Anforderungen Getränkeuntersetzer	91
38 C.5 Sicherheitsanforderungen	92

39 C.6 Systemüberwachung	93
40 C.7 Nichtfunktionale Anforderungen	93
41 C.8 Simulationsanforderungen	94
42 C.9 Anforderungen außerhalb des Projektumfangs	94
43 C.10 Zusammenfassung	95
43.0.1 Anhang D: Risikoanalyse und Testpriorisierung.	95
44 Anhang D: Risikoanalyse und Testpriorisierung	95
44.1 D.1 Zweck der Risikoanalyse	95
45 D.2 Bewertungsmethode	96
45.1 Eintrittswahrscheinlichkeit	96
45.2 Auswirkung	96
45.3 Risikopriorität	96
46 D.3 Technische Risikoanalyse	97
47 D.4 Sicherheitsrisiken	97
47.1 D.4.1 Verlust der Tischsicherheit	97
47.1.1 Beschreibung	97
47.1.2 Auswirkungen	98
47.1.3 Gegenmaßnahmen	98
47.1.4 Zugeordnete Tests	98
47.2 D.4.2 Kollision mit Objekten oder Personen	98
47.2.1 Beschreibung	98
47.2.2 Auswirkungen	98
47.2.3 Gegenmaßnahmen	98
47.2.4 Zugeordnete Tests	99
48 D.5 Softwarerisiken	99
48.1 D.5.1 Fehlerhafte Systemzustände	99
48.1.1 Beschreibung	99
48.1.2 Auswirkungen	99
48.1.3 Gegenmaßnahmen	99
48.1.4 Zugeordnete Tests	99
48.2 D.5.2 Fehlende Hardwareabstraktion	99
48.2.1 Beschreibung	99
48.2.2 Auswirkungen	100
48.2.3 Gegenmaßnahmen	100
48.2.4 Zugeordnete Tests	100

49 D.6 Projektrisiken	100
50 D.7 Testpriorisierung	100
51 D.8 Prioritätsstufen der Tests	101
51.1 Priorität P1 – Kritische Tests	101
51.2 Priorität P2 – Kernfunktionstests	101
51.3 Priorität P3 – Erweiterungstests	102
52 D.9 Maßnahmen zur Risikoreduzierung	102
52.1 Entwicklung	102
52.2 Test	102
52.3 Sicherheit	102
53 D.10 Zusammenfassung	103

1 Einleitung

1.1 Zweck der Testspezifikation

Diese Testspezifikation beschreibt die Planung, Durchführung und Bewertung der Tests für den im Rahmen der Projektarbeit entwickelten Coasterbot-Prototyp.

Ziel der Tests ist es, die im Lastenheft definierten Anforderungen systematisch zu überprüfen und nachzuweisen, dass die technische Umsetzung gemäß dem Pflichtenheft funktioniert.

Die Testspezifikation stellt eine Verbindung zwischen folgenden Dokumenten her:

- Anforderungen aus dem Lastenheft
- technischen Umsetzungen aus dem Pflichtenheft
- implementierten Systemkomponenten
- durchgeführten Prüfungen

Dadurch wird sichergestellt, dass jede relevante Funktion des Systems nachvollziehbar geprüft werden kann.

1.2 Zielsetzung der Testdurchführung

Die Testdurchführung verfolgt folgende Ziele:

- Nachweis der Funktionsfähigkeit des Coasterbot-Prototyps
- Überprüfung der autonomen Navigation
- Validierung der Hinderniserkennung und Kollisionsvermeidung
- Überprüfung der Aufnahme und Positionierung von Getränkeuntersetzern
- Prüfung sicherheitskritischer Funktionen
- Bewertung der Stabilität und Zuverlässigkeit der Software
- Überprüfung der Simulationsumgebung hinsichtlich Reproduzierbarkeit

Die Tests dienen nicht nur der Fehlererkennung, sondern auch der Bewertung der Systemqualität und der Erfüllung der Definition of Done.

1.3 Geltungsbereich

Die Testspezifikation umfasst alle Funktionen, die im Rahmen des Projekts umgesetzt werden.

Der Schwerpunkt liegt auf den Funktionen des Maturity Level 1:

- Navigation und Lokalisierung
- Hinderniserkennung

- Tischkantenerkennung
- Sicherheitsfunktionen
- Getränkeuntersetzeraufnahme
- Getränkeuntersetzerplatzierung
- Systemüberwachung
- Simulation und Testbarkeit

Funktionen höherer Reifegrade werden nicht vollständig getestet, da sie außerhalb des Projektumfangs liegen.

Dies betrifft insbesondere:

- automatisierten Getränketransport
- Bestellaufnahme
- Bezahlvorgänge
- Reinigung
- Mensch-Roboter-Interaktion
- Netzwerkkommunikation

1.4 Testobjekt

Das Testobjekt ist der Coasterbot-Prototyp, einschließlich seiner Hard- und Softwarekomponenten.

Die Tests betrachten folgende Systembestandteile:

1.4.1 Hardware

- Recheneinheit
- Sensorik
- Aktorik
- Energieversorgung
- Mechanische Konstruktion

1.4.2 Software

- Hardwareabstraktion
- Navigationskomponente
- Lokalisierung
- Bewegungssteuerung
- Untersetzerhandling
- Sicherheitslogik
- Systemüberwachung

1.4.3 Simulation

- Virtuelle Tischumgebung
- Sensormodelle
- Aktormodelle
- Simulierte Fehlerzustände
- Teststeuerung

1.5 Testgrundlagen

Die Testfälle werden aus den Anforderungen des Lastenhefts sowie den technischen Beschreibungen des Pflichtenhefts abgeleitet.

Jeder Testfall besitzt eine eindeutige Identifikation und beschreibt:

- Testziel
- Bezug zu Anforderungen
- Voraussetzungen
- Testumgebung
- Testablauf
- Eingabedaten
- Erwartetes Ergebnis
- Tatsächliches Ergebnis
- Bewertung

Die Tests werden so gestaltet, dass sie reproduzierbar durchgeführt werden können.

1.6 Teststrategie

Die Teststrategie folgt einem mehrstufigen Ansatz.

Die einzelnen Testebenen bauen aufeinander auf:

1.7 Komponententests

Komponententests prüfen einzelne Softwaremodule unabhängig voneinander.

Beispiele:

- Prüfung der Positionsberechnung
- Prüfung von Zustandsübergängen
- Prüfung der Sensorverarbeitung
- Prüfung der Fehlererkennung

Ziel ist die frühzeitige Erkennung von Fehlern innerhalb einzelner Komponenten.

1.8 Integrationstests

Integrationstests prüfen die Kommunikation zwischen mehreren Komponenten.

Beispiele:

- Übergabe von Sensordaten an die Navigation
- Übergabe von Navigationsbefehlen an die Bewegungssteuerung
- Kommunikation zwischen Sicherheitslogik und Aktorsteuerung

Ziel ist der Nachweis eines korrekten Zusammenspiels der Systemkomponenten.

1.9 Simulationstests

Simulationstests überprüfen das Verhalten des Gesamtsystems innerhalb der virtuellen Umgebung.

Dabei werden insbesondere geprüft:

- Navigation
- Hindernisvermeidung
- Tischkantenerkennung
- Untersetzerhandling
- Fehlerzustände

Die Simulation ermöglicht reproduzierbare Tests ohne Abhängigkeit von der physischen Hardware.

1.10 Systemtests

Systemtests überprüfen den vollständigen Coasterbot-Prototyp.

Dabei werden Hardware und Software gemeinsam betrachtet.

Beispiele:

- vollständiger Navigationsablauf
- Aufnahme eines Untersetzers
- Sicherheitsstopp
- Verhalten bei Fehlerzuständen

1.11 Testumgebung

Die Tests werden in mehreren Umgebungen durchgeführt.

1.11.1 Entwicklungsumgebung

Die Entwicklungsumgebung dient zur Durchführung automatisierter Softwaretests.

Sie umfasst:

- Quellcodeverwaltung
- Build-System
- Testframework
- Analysewerkzeuge

1.11.2 Simulationsumgebung

Die Simulationsumgebung ermöglicht die Prüfung des Systemverhaltens ohne reale Hardware.

Sie beinhaltet:

- virtuelle Tischumgebung
- simulierte Sensoren
- simulierte Aktoren
- definierte Testszenarien

1.11.3 Physischer Prototyp

Nach erfolgreicher Validierung in der Simulation werden ausgewählte Tests auf dem realen Prototyp durchgeführt.

Dabei werden insbesondere geprüft:

- mechanische Funktion
- reale Sensorik
- reale Bewegung
- Interaktion der Hardwarekomponenten

1.12 Bewertung von Testergebnissen

Ein Testfall wird anhand des Vergleichs zwischen erwartetem und tatsächlichem Ergebnis bewertet.

Folgende Ergebnisse sind möglich:

Ergebnis	Bedeutung
Bestanden	Erwartetes Ergebnis wurde erreicht

Ergebnis	Bedeutung
Fehlgeschlagen	Erwartetes Ergebnis wurde nicht erreicht
Nicht ausführbar	Voraussetzungen waren nicht erfüllt

Fehlgeschlagene Tests werden dokumentiert und führen zu einer Fehleranalyse sowie gegebenenfalls zu einer Anpassung der Implementierung.

1.13 Aufbau der Testspezifikation

Die weiteren Kapitel dieser Testspezifikation sind wie folgt aufgebaut:

Kapitel 2 – Testmethodik und Testmanagement Beschreibung der Testorganisation, Testprozesse und Dokumentation.

Kapitel 3 – Komponententests Beschreibung der Tests einzelner Softwaremodule.

Kapitel 4 – Integrationstests Beschreibung der Tests zwischen Softwarekomponenten.

Kapitel 5 – Simulationstests Beschreibung der virtuellen Testszenarien.

Kapitel 6 – Systemtests Beschreibung der Tests am vollständigen Prototyp.

Kapitel 7 – Testergebnisse und Bewertung Dokumentation und Auswertung der durchgeführten Tests.

Kapitel 8 – Zusammenfassung Bewertung der Testergebnisse hinsichtlich der Projektziele.

1.14 Zusammenfassung

Dieses Kapitel definiert die Grundlagen für die systematische Prüfung des Coasterbot-Prototyps.

Durch die Kombination aus Komponenten-, Integrations-, Simulations- und Systemtests wird sichergestellt, dass die entwickelten Funktionen nachvollziehbar bewertet werden können.

Die Testspezifikation bildet damit die Grundlage für den Nachweis, dass der entwickelte Prototyp die Anforderungen des Lasten- und Pflichtenhefts erfüllt.

2 Testmethodik und Testmanagement

2.1 Ziel der Testmethodik

Die Testmethodik definiert die Vorgehensweise zur Planung, Durchführung und Bewertung der Tests für den Coasterbot-Prototyp.

Ziel ist es, durch eine strukturierte und nachvollziehbare Vorgehensweise sicherzustellen, dass relevante Anforderungen geprüft werden. Die Testmethodik orientiert sich an den Prinzipien des systematischen Softwaretests und berücksichtigt sowohl die Besonderheiten eingebetteter Systeme als auch die Anforderungen an autonome Robotersysteme.

Die Testmethodik verfolgt folgende Ziele:

- vollständige Abdeckung der relevanten Anforderungen
- frühzeitige Erkennung von Fehlern
- reproduzierbare Testergebnisse
- nachvollziehbare Dokumentation
- Verifikation der technischen Umsetzung

2.2 Testvorgehensmodell

Die Entwicklung und Prüfung des Coasterbots erfolgt nach einem inkrementellen Testvorgehen.

Dabei werden Funktionen schrittweise entwickelt und anschließend auf verschiedenen Ebenen geprüft.

Der Testprozess besteht aus folgenden Phasen:

1. Testplanung
2. Testentwurf
3. Testimplementierung
4. Testdurchführung
5. Fehleranalyse
6. Testabschluss

2.2.1 Testplanung

In der Testplanung werden die Rahmenbedingungen für die Tests festgelegt.

Dazu gehören:

- Definition der Testziele
- Auswahl geeigneter Testverfahren

- Festlegung der Testumgebung
- Identifikation benötigter Testdaten
- Zuordnung der Anforderungen zu Testfällen

Die Testplanung basiert auf der Traceability-Matrix aus dem Pflichtenheft.

2.2.2 Testentwurf

Im Testentwurf werden konkrete Testfälle aus den Anforderungen abgeleitet.

Jeder Testfall erhält eine eindeutige Identifikation.

Die Testfälle werden nach folgendem Schema aufgebaut:

Feld	Beschreibung
Test-ID	Eindeutige Kennung des Testfalls
Titel	Name des Tests
Ziel	Beschreibung der zu prüfenden Funktion
Anforderungen	Zugeordnete Anforderungen
Voraussetzungen	Notwendige Ausgangsbedingungen
Testdaten	Verwendete Eingaben und Parameter
Ablauf	Beschreibung der Testschritte
Erwartetes Ergebnis	Sollverhalten
Bewertung	Testergebnis

2.3 Testebenen

Die Tests werden in verschiedene Ebenen unterteilt. Jede Ebene besitzt eine eigene Zielsetzung.

2.3.1 Komponententest

Komponententests überprüfen einzelne Softwaremodule unabhängig voneinander.

Ziel:

- korrekte Funktion einzelner Komponenten
- Prüfung interner Logik
- Fehlerisolierung

Beispiele:

- Prüfung der Lokalisierungsberechnung

- Prüfung eines Zustandsautomaten
- Prüfung der Fehlererkennung

Komponententests werden bevorzugt automatisiert durchgeführt.

2.3.2 Integrationstest

Integrationstests prüfen das Zusammenspiel mehrerer Komponenten.

Dabei werden insbesondere Schnittstellen überprüft.

Beispiele:

- Sensorabstraktion liefert korrekte Daten an Navigation
- Navigation erzeugt gültige Steuerbefehle
- Sicherheitskomponente kann Bewegungen stoppen

Ziel ist der Nachweis, dass die Komponenten gemeinsam korrekt funktionieren.

2.3.3 Simulationstest

Simulationstests prüfen das Verhalten des Gesamtsystems unter kontrollierten Bedingungen.

Die Simulation ermöglicht:

- reproduzierbare Szenarien
- sichere Prüfung kritischer Situationen
- Untersuchung verschiedener Umgebungsbedingungen

Beispiele:

- Hindernis auf der Fahrstrecke
- Verlust eines Sensorsignals
- Annäherung an eine Tischkante

2.3.4 Systemtest

Systemtests prüfen den vollständigen Prototyp.

Dabei werden alle relevanten Komponenten gemeinsam betrachtet:

- Mechanik
- Elektronik
- Software
- Sensorik
- Aktorik

Ziel ist die Überprüfung des Gesamtsystems unter realitätsnahen Bedingungen.

2.4 Testarten

Neben den Testebenen werden unterschiedliche Testarten eingesetzt.

2.4.1 Funktionstest

Funktionstests prüfen, ob das System die geforderten Funktionen korrekt ausführt.

Beispiele:

- Navigation zu einem Zielpunkt
- Aufnahme eines Getränkeuntersetzers
- Erkennen eines Hindernisses

2.4.2 Sicherheitstest

Sicherheitstests überprüfen sicherheitskritische Funktionen.

Geprüft werden:

- Not-Aus
- Tischkantenerkennung
- Kollisionsvermeidung
- Fehlerreaktionen

Diese Tests besitzen eine hohe Priorität, da Fehler zu Schäden am System oder in der Umgebung führen können.

2.4.3 Belastungstest

Belastungstests untersuchen das Verhalten unter erhöhten Anforderungen.

Beispiele:

- längere Betriebsdauer
- häufige Fahrbewegungen
- wiederholte Aufnahmevorgänge

Ziel ist die Bewertung der Stabilität und Zuverlässigkeit.

2.4.4 Regressionstest

Regressionstests stellen sicher, dass Änderungen an der Software keine bereits funktionierenden Funktionen beeinträchtigen.

Nach jeder größeren Änderung werden relevante Tests erneut ausgeführt.

2.5 Priorisierung der Testfälle

Nicht alle Anforderungen besitzen dieselbe Kritikalität. Daher werden Testfälle priorisiert.

Die Priorisierung erfolgt anhand folgender Kriterien:

- Sicherheitsrelevanz
- Bedeutung für die Kernfunktion
- Abhängigkeit anderer Komponenten
- Einfluss auf die Definition of Done

Die Prioritätsstufen sind:

Priorität	Bedeutung
P1	Kritische Funktion, muss zwingend erfüllt sein
P2	Wesentliche Funktion für den Prototyp
P3	Unterstützende oder optionale Funktion

2.5.1 Priorität P1 – Kritische Tests

Folgende Funktionen besitzen höchste Priorität:

- Erkennung von Tischkanten
- Sicherheitsstopp
- Kollisionsvermeidung
- Grundlegende Bewegungsfähigkeit
- Fehlererkennung

Ein Fehler in diesen Bereichen verhindert einen sicheren Betrieb des Roboters.

2.5.2 Priorität P2 – Kernfunktionen

Folgende Funktionen sind für die eigentliche Aufgabe des Prototyps erforderlich:

- Navigation (zu Zielpunkten)
- Lokalisierung
- Untersetzeraufnahme
- Untersetzerplatzierung
- Systemüberwachung

2.5.3 Priorität P3 – Erweiterungen

Diese Funktionen werden nachrangig betrachtet:

- (Erweiterte) Simulationen
- Optimierungen
- Vorbereitung zukünftiger Ausbaustufen

2.6 Testdaten und Testszenarien

Die Testdaten werden so gewählt, dass typische sowie kritische Situationen abgedeckt werden.

Beispiele:

2.6.1 Normalszenario

- Freie Tischfläche
- Keine Hindernisse
- Ausreichender Energiezustand

Ziel:

Überprüfung des normalen Betriebs.

2.6.2 Hindernisszenario

- Objekt befindet sich auf geplanter Route

Ziel:

Prüfung der Hinderniserkennung und Navigation.

2.6.3 Grenzszenario

- Roboter nähert sich Tischkante
- Sensor liefert kritische Werte

Ziel:

Überprüfung der Sicherheitsfunktionen.

2.6.4 Fehlerszenario

- Sensor fällt aus
- Aktor reagiert nicht

Ziel:

Überprüfung der Fehlerbehandlung.

2.7 Testautomatisierung

Ein wesentlicher Bestandteil des Testkonzepts ist die Automatisierung wiederkehrender Prüfungen.

Automatisierte Tests ermöglichen:

- schnelle Wiederholung von Testfällen
- reproduzierbare Ergebnisse
- frühzeitige Fehlererkennung
- Unterstützung der Softwareentwicklung

Automatisiert werden insbesondere:

- Komponententests
- Schnittstellentests
- Simulationstests
- Zustandsübergänge

2.8 Fehlerverwaltung

Festgestellte Fehler werden strukturiert dokumentiert.

Jeder Fehler enthält:

Feld	Beschreibung
Fehler-ID	Eindeutige Kennung
Beschreibung	Fehlerbeschreibung
Betroffene Komponente	Verantwortliches Modul
Priorität	Kritikalität
Reproduzierbarkeit	Möglichkeit der Wiederholung
Status	Bearbeitungsstand

Mögliche Fehlerstatus:

Status	Bedeutung
Neu	Fehler wurde erkannt
Analyse	Ursache wird untersucht
Behoben	Fehler wurde korrigiert
Verifiziert	Korrektur wurde getestet

2.9 Testabschluss

Ein Testzyklus gilt als abgeschlossen, wenn:

- alle geplanten Tests durchgeführt wurden
- Ergebnisse dokumentiert wurden
- kritische Fehler behoben wurden
- die Testabdeckung bewertet wurde

Die Ergebnisse werden in einem Testbericht zusammengefasst.

Dieser enthält:

- Anzahl ausgeführter Tests
- erfolgreiche Tests
- fehlgeschlagene Tests
- offene Probleme
- Bewertung des Systemzustands

2.10 Zusammenfassung

Die Testmethodik definiert einen strukturierten Prozess zur Überprüfung des Coasterbot-Prototyps.

Durch die Kombination verschiedener Testebenen und Testarten wird sichergestellt, dass sowohl einzelne Softwarekomponenten als auch das Gesamtsystem überprüft werden.

Die Priorisierung der Tests stellt sicher, dass sicherheitskritische und für die Kernfunktion notwendige Anforderungen zuerst validiert werden.

Damit bildet dieses Testmanagement die Grundlage für die nachfolgenden detaillierten Testfälle.

3 Komponententests

3.1 Ziel der Komponententests

Komponententests überprüfen einzelne Softwaremodule des Coasterbot-Systems unabhängig von anderen Systembestandteilen. Ziel ist es, die korrekte Funktion der einzelnen Komponenten sicherzustellen, bevor diese in das Gesamtsystem integriert werden.

Durch die isolierte Prüfung einzelner Komponenten können Fehler frühzeitig erkannt und deren Ursachen eindeutig zugeordnet werden.

Die Komponententests konzentrieren sich auf die im Pflichtenheft definierten Softwaremodule:

- Hardwareabstraktion
- Lokalisierung
- Navigation
- Bewegungssteuerung
- Untersetzerhandling
- Sicherheitskomponente
- Systemüberwachung

Die Tests werden bevorzugt automatisiert ausgeführt, um eine wiederholbare und effiziente Qualitätssicherung zu ermöglichen.

3.2 Testumgebung für Komponententests

Die Komponententests werden ohne vollständige Hardware durchgeführt. Hardwareabhängige Funktionen werden durch simulierte Schnittstellen ersetzt.

Die Testumgebung besteht aus:

- Softwarekomponenten des Coasterbots
- Simulierten Sensordaten
- Simulierten Aktorzuständen
- Testframework
- Testdatenverwaltung

Die Hardwareabstraktion ermöglicht es, einzelne Komponenten unabhängig von realen Sensoren und Aktoren zu prüfen.

3.3 Testfallstruktur

Alle Komponententests werden nach einer einheitlichen Struktur dokumentiert.

Jeder Testfall enthält:

Feld	Beschreibung
Test-ID	Eindeutige Kennung
Komponente	Getestetes Modul
Ziel	Zweck des Tests
Anforderungen	Zugeordnete Lastenheft-Anforderungen
Voraussetzungen	Notwendige Bedingungen
Eingaben	Testdaten
Ablauf	Durchführung
Erwartetes Ergebnis	Sollverhalten
Bewertung	Testergebnis

3.4 Komponententest: Hardwareabstraktion

3.4.1 Testfall CT-HW-001: Verarbeitung von Sensordaten

Ziel Überprüfung, ob die Hardwareabstraktion Sensordaten korrekt einliest und für die darüberliegenden Softwarekomponenten bereitstellt.

Zugeordnete Anforderungen

- NFR-008: Software muss Hardwareabstraktion unterstützen
- NFR-007: Sensoren und Aktoren müssen austauschbar sein

Voraussetzungen

- Simulierter Sensor ist verfügbar
- Hardwareabstraktionsschnittstelle ist initialisiert

Eingaben Simulierter Abstandswert:

Entfernung = 500 mm

Ablauf

1. Simulierter Sensor liefert Messwert
2. Hardwareabstraktion empfängt den Messwert
3. Daten werden in internes Format übertragen
4. Navigationskomponente ruft Sensordaten ab

Erwartetes Ergebnis

- Sensordaten werden korrekt verarbeitet
- Datenformat entspricht der definierten Schnittstelle
- Navigation erhält gültige Informationen

Bewertung Bestanden, wenn die übertragenen Daten mit den Eingangsdaten übereinstimmen.

3.5 Komponententest: Lokalisierung

3.5.1 Testfall CT-LOC-001: Positionsaktualisierung

Ziel Überprüfung der Berechnung und Aktualisierung der Roboterposition.

Zugeordnete Anforderungen

- NAV-006: Der Bot muss seine Position bestimmen können
- NAV-007: Der Bot muss seine Orientierung bestimmen können

Voraussetzungen

- Lokalisierungskomponente ist aktiv
- Startposition ist definiert

Eingaben Startposition:

$$(x, y, \theta) = (0, 0, 0)$$

Bewegungsänderung:

$$\Delta x = 100mm$$

Ablauf

1. Startposition wird gesetzt
2. Bewegungsinformation wird übergeben
3. Lokalisierung verarbeitet die Daten
4. Neue Position wird berechnet

Erwartetes Ergebnis Die Position wird korrekt aktualisiert:

$$(x, y, \theta) = (100, 0, 0)$$

Bewertung Bestanden, wenn die berechnete Position innerhalb der definierten Genauigkeit liegt.

3.6 Komponententest: Navigation

3.6.1 Testfall CT-NAV-001: Berechnung einer gültigen Route

Ziel Überprüfung, ob die Navigationskomponente eine gültige Route zwischen Start- und Zielpunkt erzeugen kann.

Zugeordnete Anforderungen

- NAV-005: Der Bot muss seine Fahrtroute dynamisch anpassen
- NAV-008: Der Bot muss seinen Zielpunkt erreichen

Voraussetzungen

- Startposition bekannt
- Zielposition bekannt
- Keine Hindernisse vorhanden

Eingaben Start:

$(0, 0)$

Ziel:

$(1000, 500)$

Ablauf

1. Startposition wird gesetzt
2. Zielposition wird übergeben
3. Navigation berechnet Route
4. Route wird ausgegeben

Erwartetes Ergebnis

- Eine gültige Route wird erzeugt
- Zielpunkt ist erreichbar
- Route enthält keine ungültigen Bewegungen

Bewertung Bestanden, wenn eine gültige Bewegungssequenz erzeugt wird.

3.6.2 Testfall CT-NAV-002: Hinderniserkennung

Ziel Überprüfung der Reaktion der Navigation auf ein erkanntes Hindernis.

Zugeordnete Anforderungen

- NAV-001: Hindernisse erkennen
- NAV-002: Hindernissen ausweichen

Voraussetzungen

- Navigationsziel ist gesetzt
- Hindernisdaten werden simuliert

Eingaben Hindernisposition:

(500, 0)

Ablauf

1. Navigation startet
2. Hindernis wird erkannt
3. Route wird neu berechnet

Erwartetes Ergebnis

- Hindernis wird erkannt
- Neue Route vermeidet das Hindernis
- Keine Kollision entsteht

Bewertung Bestanden, wenn eine kollisionsfreie Route erzeugt wird.

3.7 Komponententest: Bewegungssteuerung

3.7.1 Testfall CT-MOT-001: Umsetzung von Bewegungsbefehlen

Ziel Überprüfung, ob Bewegungsbefehle korrekt in Aktorbefehle umgesetzt werden.

Zugeordnete Anforderungen

- NAV-009: Der Bot muss sich kontrolliert bewegen
- SAF-004: Der Bot darf nicht mit hoher Geschwindigkeit kollidieren

Voraussetzungen

- Motorsteuerung ist initialisiert

Eingaben Bewegungsbefehl:

Geschwindigkeit: 100 mm/s

Richtung: vorwärts

Ablauf

1. Bewegungsbefehl wird übergeben
2. Steuerung berechnet Aktorsignale
3. Motorzustand wird geprüft

Erwartetes Ergebnis

- Motoren erhalten korrekte Steuerwerte
- Geschwindigkeit entspricht Vorgabe
- Sicherheitsgrenzen werden eingehalten

Bewertung Bestanden, wenn die Ausgangswerte korrekt erzeugt werden.

3.8 Komponententest: Untersetzerhandling

3.8.1 Testfall CT-CST-001: Aufnahme eines Getränkeuntersetzers

Ziel Überprüfung des Aufnahmeprozesses.

Zugeordnete Anforderungen

- CST-001: Untersetzer aufnehmen
- CST-005: Erfolgreiche Aufnahme erkennen

Voraussetzungen

- Untersetzer befindet sich an definierter Position
- Aufnahmemechanismus ist verfügbar

Eingaben Untersetzerstatus:

Verfügbar = TRUE

Ablauf

1. Aufnahmeprozess wird gestartet
2. Aktor wird aktiviert
3. Sensorstatus wird geprüft

Erwartetes Ergebnis

- Aufnahme wird durchgeführt
- Erfolgreiche Aufnahme wird erkannt
- Zustand wechselt zu “Untersetzer aufgenommen”

Bewertung Bestanden, wenn der Zustand korrekt aktualisiert wird.

3.8.2 Testfall CT-CST-002: Fehler bei Untersetzeraufnahme

Ziel Überprüfung der Fehlererkennung während der Aufnahme.

Zugeordnete Anforderungen

- CST-005: Aufnahme erkennen
- MON-005: Fehlgeschlagene Aufgaben melden

Voraussetzungen

- Aufnahmevorgang gestartet

Eingaben Simulierter Fehler:

Untersetzer erkannt = FALSE

Ablauf

1. Aufnahme wird gestartet
2. Sensor meldet fehlende Aufnahme
3. System verarbeitet Fehler

Erwartetes Ergebnis

- Fehler wird erkannt
- Aufgabe wird abgebrochen
- Fehler wird gemeldet

Bewertung Bestanden, wenn der Fehler korrekt verarbeitet wird.

3.9 Komponententest: Sicherheitskomponente

3.9.1 Testfall CT-SAF-001: Sicherheitsstopp

Ziel Überprüfung der Reaktion auf eine Gefahrensituation.

Zugeordnete Anforderungen

- SAF-001: Bei Gefahr sofort anhalten
- SAF-006: Not-Aus unterstützen

Voraussetzungen

- Roboter befindet sich im Bewegungszustand

Eingaben Not-Aus-Signal:

EmergencyStop = TRUE

Ablauf

1. Bewegungssteuerung ist aktiv
2. Sicherheitskomponente erhält Stoppsignal
3. Sicherheitszustand wird aktiviert

Erwartetes Ergebnis

- Bewegungsbefehle werden unterbrochen
- Roboter wechselt in sicheren Zustand
- Status wird aktualisiert

Bewertung Bestanden, wenn der Stopp innerhalb der definierten Reaktionszeit erfolgt.

3.10 Komponententest: Systemüberwachung

3.10.1 Testfall CT-MON-001: Fehlererkennung und Protokollierung

Ziel Überprüfung der Diagnosefunktion.

Zugeordnete Anforderungen

- MON-001: Fehler erkennen
- MON-002: Fehler protokollieren

Voraussetzungen

- Systemüberwachung ist aktiv

Eingaben Simulierter Fehler:

Sensorfehler = TRUE

Ablauf

1. Fehler wird erzeugt
2. Systemüberwachung verarbeitet Ereignis
3. Fehler wird gespeichert

Erwartetes Ergebnis

- Fehler wird erkannt
- Fehler wird protokolliert
- Status wird aktualisiert

Bewertung Bestanden, wenn der Fehler vollständig dokumentiert wurde.

3.11 Zusammenfassung der Komponententests

Die Komponententests überprüfen die grundlegenden Funktionen der einzelnen Softwaremodule des Coasterbots.

Durch die isolierte Prüfung werden Fehler frühzeitig erkannt und die Grundlage für nachfolgende Integrations- und Systemtests geschaffen.

Die wichtigsten überprüften Bereiche sind:

- Hardwareabstraktion
- Navigation
- Lokalisierung
- Bewegungssteuerung
- Untersetzerhandling
- Sicherheit
- Systemüberwachung

Die erfolgreichen Komponententests bilden die Voraussetzung für die Integration der einzelnen Softwarebestandteile zu einem vollständigen Robotersystem.

4 Integrationstests

4.1 Ziel der Integrationstests

Integrationstests überprüfen das Zusammenspiel mehrerer Softwarekomponenten des Coasterbot-Systems. Während Komponententests einzelne Module isoliert betrachten, wird bei Integrationstests die korrekte Kommunikation und Zusammenarbeit zwischen den Komponenten untersucht.

Ziel der Integrationstests ist es, sicherzustellen, dass die einzelnen Systembestandteile gemeinsam die geforderten Funktionen erfüllen.

Der Fokus liegt dabei auf:

- korrekter Datenübertragung zwischen Komponenten
- definierten Schnittstellen
- korrekten Zustandsübergängen
- fehlerfreier Zusammenarbeit von Hard- und Softwareabstraktion
- sicherem Verhalten bei kritischen Situationen

Die Integrationstests bilden die Grundlage für die anschließenden Systemtests, bei denen der vollständige Prototyp betrachtet wird.

4.2 Testumgebung für Integrationstests

Die Integrationstests werden in einer Umgebung durchgeführt, die mehrere reale oder simulierte Komponenten kombiniert.

Die Testumgebung umfasst:

- Softwarekomponenten des Coasterbots
- Hardwareabstraktionsschicht
- simulierte Sensordaten
- simulierte Aktorzustände
- Simulationsumgebung
- Testframework

Je nach Testfall können einzelne Hardwarekomponenten durch Simulationsmodelle ersetzt werden.

Dadurch kann das Verhalten der Software unabhängig von der finalen Hardware überprüft werden.

4.3 Testfallstruktur

Die Integrationstestfälle werden nach folgendem Schema beschrieben:

Feld	Beschreibung
Test-ID	Eindeutige Identifikation
Testname	Bezeichnung des Tests
Ziel	Zu überprüfende Integration
Anforderungen	Zugeordnete Anforderungen
Beteiligte Komponenten	Verwendete Systemmodule
Voraussetzungen	Ausgangszustand
Ablauf	Testschritte
Erwartetes Ergebnis	Sollverhalten
Bewertung	Testergebnis

4.4 Integrationstest: Sensorik und Navigation

4.4.1 Testfall IT-SEN-NAV-001: Verarbeitung von Hindernisdaten

Ziel Überprüfung der korrekten Übergabe von Sensordaten an die Navigationskomponente.

Zugeordnete Anforderungen

- NAV-001: Der Bot muss Hindernisse erkennen
- NAV-002: Der Bot muss Hindernissen ausweichen
- NFR-008: Software muss Hardwareabstraktion unterstützen

Beteiligte Komponenten

- Sensorabstraktion
- Hinderniserkennung
- Navigation

Voraussetzungen

- Navigation ist aktiv
- Simulierter Abstandssensor ist verfügbar
- Eine Route zu einem Zielpunkt wurde erstellt

Eingaben Sensordaten:

Hindernis erkannt = TRUE

Abstand = 200 mm

Position = (500,300)

Ablauf

1. Sensorabstraktion empfängt Messwerte
2. Messwerte werden an die Hinderniserkennung übertragen
3. Hinderniserkennung bewertet die Situation
4. Navigationskomponente erhält die Information über das Hindernis
5. Neue Route wird berechnet

Erwartetes Ergebnis

- Sensordaten werden korrekt übertragen
- Hindernis wird erkannt
- Navigation reagiert auf die Änderung
- Neue kollisionsfreie Route wird erzeugt

Bewertung Bestanden, wenn die Navigation innerhalb der definierten Reaktionszeit eine gültige Anpassung erzeugt.

4.5 Integrationstest: Lokalisierung und Navigation

4.5.1 Testfall IT-LOC-NAV-001: Zielnavigation mit Positionsdaten

Ziel Überprüfung, ob die Navigation die aktuelle Position korrekt verwendet.

Zugeordnete Anforderungen

- NAV-006: Der Bot muss seine Position bestimmen können
- NAV-007: Der Bot muss seine Orientierung bestimmen können
- NAV-008: Der Bot muss seinen Zielpunkt erreichen

Beteiligte Komponenten

- Lokalisierung
- Navigation
- Bewegungssteuerung

Voraussetzungen

- Roboterposition ist initialisiert
- Zielpunkt ist bekannt

Eingaben Startposition:

$x = 0 \text{ mm}$

$y = 0 \text{ mm}$

Orientierung = 0°

Ziel:

$x = 1000 \text{ mm}$

$y = 500 \text{ mm}$

Ablauf

1. Lokalisierung stellt aktuelle Position bereit
2. Navigation berechnet Bewegungsrichtung
3. Bewegungssteuerung erhält Fahrbefehle
4. Positionsänderungen werden zurückgemeldet
5. Navigation aktualisiert die Route

Erwartetes Ergebnis

- Navigation verwendet aktuelle Positionsdaten
- Fahrbefehle werden korrekt erzeugt
- Roboter bewegt sich in Richtung Ziel
- Ziel wird innerhalb der definierten Genauigkeit erreicht

Bewertung Bestanden, wenn die Positionsabweichung kleiner als der definierte Grenzwert ist.

4.6 Integrationstest: Navigation und Bewegungssteuerung

4.6.1 Testfall IT-NAV-MOT-001: Umsetzung von Navigationsbefehlen

Ziel Überprüfung der Schnittstelle zwischen Navigation und Motorsteuerung.

Zugeordnete Anforderungen

- NAV-009: Der Bot muss sich kontrolliert bewegen
- SAF-004: Der Bot darf nicht mit hoher Geschwindigkeit in Hindernisse fahren

Beteiligte Komponenten

- Navigation
- Bewegungssteuerung
- Hardwareabstraktion

Voraussetzungen

- Route wurde berechnet
- Motorsteuerung ist betriebsbereit

Eingaben Navigationsbefehl:

Geschwindigkeit = 100 mm/s

Richtung = 45°

Distanz = 500 mm

Ablauf

1. Navigation erzeugt Bewegungsbefehl
2. Bewegungssteuerung verarbeitet den Befehl
3. Aktorschnittstelle erzeugt Steuersignale
4. Bewegungsstatus wird zurückgemeldet

Erwartetes Ergebnis

- Bewegungsbefehle werden korrekt umgesetzt
- Geschwindigkeit bleibt innerhalb definierter Grenzen
- Bewegungsstatus wird aktualisiert

Bewertung Bestanden, wenn die erzeugte Bewegung dem Navigationsauftrag entspricht.

4.7 Integrationstest: Sicherheitslogik und Bewegungssteuerung

4.7.1 Testfall IT-SAF-MOT-001: Unterbrechen einer Bewegung bei Gefahr

Ziel Überprüfung, ob die Sicherheitskomponente jederzeit Einfluss auf die Bewegungssteuerung nehmen kann.

Zugeordnete Anforderungen

- SAF-001: Der Bot muss bei Gefahr sofort anhalten
- SAF-003: Der Bot darf den Tisch nicht verlassen
- SAF-006: Der Bot muss einen Not-Aus unterstützen

Beteiligte Komponenten

- Sicherheitskomponente
- Bewegungssteuerung
- Aktorsteuerung

Voraussetzungen

- Roboter befindet sich in Bewegung
- Sicherheitsüberwachung ist aktiv

Eingaben Gefahrensignal:

Tischkante erkannt = TRUE

Ablauf

1. Roboter fährt eine definierte Route
2. Sensorik meldet kritische Situation
3. Sicherheitskomponente erzeugt Stoppsignal
4. Bewegungssteuerung beendet Bewegung

Erwartetes Ergebnis

- Bewegung wird sofort beendet
- Aktoren wechseln in sicheren Zustand
- Fehler- oder Warnstatus wird erzeugt

Bewertung Bestanden, wenn keine weitere Bewegung nach Aktivierung der Sicherheitsfunktion erfolgt.

4.8 Integrationstest: Untersetzerhandling und Navigation

4.8.1 Testfall IT-NAV-CST-001: Navigation zu einem Untersetzer und Aufnahme

Ziel Überprüfung des Zusammenspiels zwischen Navigation und Untersetzerhandling.

Zugeordnete Anforderungen

- CST-001: Der Bot muss einen Getränkeuntersetzer aufnehmen
- CST-005: Der Bot muss erkennen, ob ein Untersetzer aufgenommen wurde
- NAV-008: Der Bot muss Zielpunkte erreichen

Beteiligte Komponenten

- Navigation
- Lokalisierung
- Untersetzerhandling
- Systemüberwachung

Voraussetzungen

- Untersetzterposition ist bekannt
- Roboter befindet sich in Startposition

Eingaben Untersetzterposition:

x = 700 mm

y = 400 mm

Ablauf

1. Zielposition des Untersetzers wird übergeben
2. Navigation berechnet Route
3. Roboter fährt zur Position
4. Aufnahmeprozess wird gestartet
5. Sensor überprüft Aufnahme

Erwartetes Ergebnis

- Roboter erreicht Untersetzterposition
- Aufnahme wird durchgeführt
- Erfolgreiche Aufnahme wird erkannt

Bewertung Bestanden, wenn der vollständige Ablauf ohne Fehler abgeschlossen wird.

4.9 Integrationstest: Systemüberwachung und Komponenten

4.9.1 Testfall IT-MON-001: Fehlerweitergabe im Gesamtsystem

Ziel Überprüfung der Weitergabe und Verarbeitung von Fehlerzuständen.

Zugeordnete Anforderungen

- MON-001: Der Bot muss Fehler erkennen
- MON-002: Der Bot muss Fehler protokollieren
- MON-005: Der Bot muss fehlgeschlagene Aufgaben melden

Beteiligte Komponenten

- Sensorik
- Systemüberwachung
- Sicherheitskomponente
- Benutzerstatusanzeige

Voraussetzungen

- System befindet sich im Betriebszustand

Eingaben Simulierter Fehler:

Sensorfehler = TRUE

Ablauf

1. Sensor meldet Fehler
2. Systemüberwachung erkennt Fehler
3. Fehler wird gespeichert
4. Sicherheitskomponente bewertet Zustand
5. Statusmeldung wird erzeugt

Erwartetes Ergebnis

- Fehler wird erkannt
- Fehler wird protokolliert
- System wechselt gegebenenfalls in sicheren Zustand
- Benutzer erhält Statusinformation

Bewertung Bestanden, wenn der Fehler vollständig verarbeitet wird.

4.10 Integrationstest: Hardwareabstraktion und Simulation

4.10.1 Testfall IT-HW-SIM-001: Austausch realer Hardware durch Simulation

Ziel Überprüfung, ob Softwarekomponenten ohne Anpassung mit simulierten Hardwarekomponenten betrieben werden können.

Zugeordnete Anforderungen

- NFR-004: Software muss simuliert werden können
- NFR-007: Sensoren und Aktoren müssen austauschbar sein

Beteiligte Komponenten

- Hardwareabstraktion
- Simulationsmodelle
- Robotersoftware

Voraussetzungen

- Simulationsumgebung ist gestartet
- Virtuelle Sensoren und Aktoren sind verfügbar

Ablauf

1. Reale Hardwaretreiber werden durch Simulationsmodelle ersetzt
2. Software wird gestartet
3. Testablauf wird ausgeführt
4. Datenfluss wird überprüft

Erwartetes Ergebnis

- Software startet ohne Anpassungen
- Simulierte Sensorwerte werden verarbeitet
- Aktorbefehle werden korrekt erzeugt

Bewertung Bestanden, wenn die Software vollständig in der Simulation betrieben werden kann.

4.11 Zusammenfassung der Integrationstests

Die Integrationstests überprüfen die Zusammenarbeit der zentralen Systemkomponenten des Coasterbots.

Die wichtigsten geprüften Integrationen sind:

- Sensorik und Navigation
- Lokalisierung und Navigation
- Navigation und Bewegungssteuerung
- Sicherheitslogik und Akteursteuerung

- Navigation und Untersetzerhandling
- Systemüberwachung und Fehlerbehandlung
- Hardwareabstraktion und Simulation

Durch erfolgreiche Integrationstests wird sichergestellt, dass die einzelnen Softwaremodule nicht nur isoliert funktionieren, sondern als zusammenhängendes System die Anforderungen des Pflichtenhefts erfüllen.

5 Simulationstests

5.1 Ziel der Simulationstests

Simulationstests dienen der Überprüfung des Verhaltens des Coasterbot-Systems innerhalb einer virtuellen Umgebung. Sie ermöglichen die Validierung von Softwarefunktionen, bevor diese auf der realen Hardware ausgeführt werden.

Die Simulation bildet dabei die für den Betrieb relevanten Eigenschaften des Systems nach:

- Tischumgebung
- Roboterbewegung
- Sensorverhalten
- Aktorverhalten
- Hindernisse
- Fehlerzustände

Das Ziel besteht darin, nachzuweisen, dass die Software auch unter kontrollierten und reproduzierbaren Bedingungen die erwarteten Systemreaktionen zeigt.

Die Simulationstests erfüllen insbesondere folgende Aufgaben:

- Überprüfung autonomer Navigation
- Validierung der Sicherheitsfunktionen
- Prüfung der Fehlerbehandlung
- Nachweis der Reproduzierbarkeit von Fahrmanövern
- Unterstützung automatisierter Tests

5.2 Testumgebung der Simulation

Die Simulation basiert auf der im Pflichtenheft beschriebenen Simulationsarchitektur.

Die Testumgebung besteht aus folgenden Komponenten:

Komponente	Aufgabe
Simulationsumgebung	Abbildung der virtuellen Tischumgebung
Robotermodell	Simulation der Bewegung und Position
Sensormodelle	Erzeugung virtueller Messwerte
Aktormodelle	Simulation der Motor- und Mechanikbefehle
Teststeuerung	Automatisierte Durchführung von Szenarien
Testauswertung	Vergleich von Soll- und Istverhalten

Die Software des Coasterbots wird dabei möglichst unverändert aus der realen Umgebung übernommen. Lediglich hardwarenahe Komponenten werden durch Simulationsmodelle ersetzt.

5.3 Anforderungen an Simulationstests

Die Simulationstests basieren auf den im Pflichtenheft definierten Simulationsanforderungen.

Folgende Eigenschaften müssen erfüllt werden:

Anforderung	Beschreibung
Reproduzierbarkeit	Gleiche Eingaben erzeugen gleiche Ergebnisse
Sensorabstraktion	Sensorwerte können simuliert werden
Aktorabstraktion	Aktorverhalten kann modelliert werden
Fehlerfähigkeit	Fehlerzustände können gezielt erzeugt werden
Testunterstützung	Alle definierten Testfälle können ausgeführt werden
Validierung	Simuliertes Verhalten entspricht erwarteten Verhalten

5.4 Simulationstest: Grundlegende Navigation

5.4.1 Testfall ST-SIM-001: Navigation zu einem Zielpunkt

Ziel Überprüfung der grundlegenden Navigationsfähigkeit innerhalb der simulierten Tischumgebung.

Zugeordnete Anforderungen

- NAV-006: Der Bot muss seine Position bestimmen können
- NAV-008: Der Bot muss seinen Zielpunkt erreichen
- SIM-001: Fahrmanöver müssen reproduzierbar sein

Simulationsumgebung

- Tischgröße: definiert
- Keine Hindernisse vorhanden
- Roboterposition bekannt
- Zielposition bekannt

Voraussetzungen

- Simulation ist gestartet
- Robotermodell ist initialisiert
- Navigationskomponente ist aktiv

Eingaben Startposition:

$x = 0 \text{ mm}$

$y = 0 \text{ mm}$

Orientierung = 0°

Zielposition:

$x = 1000 \text{ mm}$

$y = 500 \text{ mm}$

Ablauf

1. Simulation wird gestartet
2. Roboter wird an Startposition platziert
3. Zielpunkt wird an Navigation übergeben
4. Navigation berechnet Route
5. Bewegungssteuerung führt Fahrbefehle aus
6. Position wird während der Bewegung überwacht

Erwartetes Ergebnis

- Roboter bewegt sich entlang einer gültigen Route
- Zielpunkt wird erreicht
- Keine unerwarteten Bewegungen treten auf
- Ergebnis ist bei Wiederholung identisch

Bewertung Bestanden, wenn der Zielpunkt innerhalb der definierten Genauigkeit erreicht wird.

5.5 Simulationstest: Hinderniserkennung und Ausweichverhalten

5.5.1 Testfall ST-SIM-002: Dynamische Hindernisvermeidung

Ziel Überprüfung, ob der Roboter Hindernisse erkennt und seine Route selbstständig anpasst.

Zugeordnete Anforderungen

- NAV-001: Hindernisse erkennen
- NAV-002: Hindernissen ausweichen
- NAV-005: Fahrtroute dynamisch anpassen
- SIM-004: Hindernisse müssen berücksichtigt werden

Simulationsumgebung

- Tischfläche vorhanden
- Hindernis wird auf geplanter Route platziert
- Zielpunkt erreichbar

Voraussetzungen

- Navigation ist aktiv
- Hindernissensoren funktionieren

Eingaben Hindernis:

Position = (500 mm, 250 mm)

Größe = 100 mm x 100 mm

Ablauf

1. Navigation startet Fahrt zum Ziel
2. Roboter nähert sich Hindernis
3. Sensorsimulation erzeugt Hinderniserkennung
4. Navigation berechnet neue Route
5. Bewegung wird fortgesetzt

Erwartetes Ergebnis

- Hindernis wird erkannt
- Kollision wird verhindert
- Neue Route wird berechnet
- Zielpunkt wird weiterhin erreicht

Bewertung Bestanden, wenn der Roboter das Hindernis vollständig umgeht.

5.6 Simulationstest: Tischkantenerkennung

5.6.1 Testfall ST-SIM-003: Verhindern des Verlassens der Tischfläche

Ziel Überprüfung der Sicherheitsreaktion bei Annäherung an eine Tischkante.

Zugeordnete Anforderungen

- NAV-003: Tischende erkennen
- NAV-004: Auf dem Tisch bleiben
- SAF-003: Tisch nicht verlassen

Simulationsumgebung

- Tischrand wird modelliert
- Roboter fährt kontrolliert Richtung Rand

Voraussetzungen

- Tischkantensensor ist aktiv
- Sicherheitslogik ist aktiviert

Ablauf

1. Roboter bewegt sich Richtung Tischkante
2. Simulation erzeugt kritische Abstandswerte
3. Sicherheitskomponente verarbeitet Signal
4. Bewegungssteuerung erhält Stoppsignal

Erwartetes Ergebnis

- Tischkante wird erkannt
- Roboter stoppt vor dem Rand
- Tischfläche wird nicht verlassen

Bewertung Bestanden, wenn der Roboter sicher innerhalb der Tischfläche verbleibt.

5.7 Simulationstest: Reproduzierbarkeit von Fahrmanövern

5.7.1 Testfall ST-SIM-004: Wiederholbarkeit identischer Bewegungsabläufe

Ziel Überprüfung, ob gleiche Eingangsbedingungen zu reproduzierbaren Ergebnissen führen.

Zugeordnete Anforderungen

- SIM-001: Fahrmanöver müssen reproduzierbar sein
- SIM-007: Simulationsverhalten muss erwartbarem Verhalten entsprechen

Voraussetzungen

- Identisches Szenario kann mehrfach gestartet werden

Ablauf

1. Simulation wird mit definierten Parametern gestartet
2. Navigation wird ausgeführt
3. Bewegungsverlauf wird gespeichert
4. Simulation wird erneut mit gleichen Parametern gestartet
5. Ergebnisse werden verglichen

Erwartetes Ergebnis

- Fahrwege stimmen überein
- Zustandsübergänge erfolgen identisch
- Ergebnisse sind vergleichbar

Bewertung Bestanden, wenn die Abweichungen innerhalb definierter Grenzen liegen.

5.8 Simulationstest: Sensorausfall

5.8.1 Testfall ST-SIM-005: Reaktion auf fehlerhafte Sensordaten

Ziel Überprüfung des Systemverhaltens bei einem simulierten Sensorausfall.

Zugeordnete Anforderungen

- SAF-002: Bei Sensorfehler sicheren Zustand einnehmen
- MON-001: Fehler erkennen
- MON-002: Fehler protokollieren
- SIM-005: Fehlersituationen simulieren können

Voraussetzungen

- Roboter befindet sich im Fahrbetrieb
- Fehlersimulation ist aktiviert

Eingaben Fehler:

Abstandssensor = nicht verfügbar

Ablauf

1. Simulation startet normalen Betrieb
2. Sensorfehler wird ausgelöst
3. Systemüberwachung erkennt Fehler
4. Sicherheitskomponente bewertet Zustand

Erwartetes Ergebnis

- Fehler wird erkannt
- Fehler wird protokolliert
- Roboter stoppt oder wechselt in sicheren Zustand
- Statusmeldung wird erzeugt

Bewertung Bestanden, wenn das System keine unsichere Bewegung fortsetzt.

5.9 Simulationstest: Aktorfehler

5.9.1 Testfall ST-SIM-006: Verhalten bei fehlerhafter Motorsteuerung

Ziel Überprüfung der Fehlerbehandlung bei einem Aktorausfall.

Zugeordnete Anforderungen

- MON-001: Fehler erkennen
- MON-005: Fehlgeschlagene Aufgaben melden
- SIM-003: Aktoren können simuliert werden

Voraussetzungen

- Roboter befindet sich in Bewegung

Eingaben Fehler:

Motor links = ausgefallen

Ablauf

1. Bewegungsbefehl wird erzeugt
2. Simulierter Aktor reagiert fehlerhaft
3. Systemüberwachung erkennt Abweichung
4. Sicherheitslogik wird aktiviert

Erwartetes Ergebnis

- Aktorfehler wird erkannt
- Bewegung wird kontrolliert beendet
- Fehler wird gemeldet

Bewertung Bestanden, wenn das System sicher reagiert.

5.10 Simulationstest: Untersetzerhandling

5.10.1 Testfall ST-SIM-007: Aufnahme und Platzierung eines Untersetzers

Ziel Überprüfung des vollständigen Untersetzerprozesses innerhalb der Simulation.

Zugeordnete Anforderungen

- CST-001: Untersetzer aufnehmen
- CST-003: Untersetzer platzieren
- CST-005: Aufnahme erkennen
- SIM-006: Simulation unterstützt Testfälle

Voraussetzungen

- Untersetzer ist virtuell vorhanden
- Zielposition ist definiert

Ablauf

1. Roboter navigiert zum Untersetzer
2. Aufnahmevorgang wird gestartet
3. Zustand wird überprüft
4. Roboter fährt zur Zielposition
5. Untersetzer wird platziert

Erwartetes Ergebnis

- Untersetzer wird erkannt
- Aufnahme funktioniert
- Transport erfolgt
- Platzierung wird bestätigt

Bewertung Bestanden, wenn der komplette Prozess ohne Fehler abgeschlossen wird.

5.11 Simulationstest: Validierung Simulation gegen erwartetes Verhalten

5.11.1 Testfall ST-SIM-008: Vergleich Simulation und Systemverhalten

Ziel Überprüfung, ob die Simulation das erwartete Verhalten des realen Systems ausreichend genau abbildet.

Zugeordnete Anforderungen

- SIM-007: Simulationsverhalten entspricht erwartetem Verhalten

Ablauf

1. Test wird zunächst in der Simulation durchgeführt
2. Ergebnis wird dokumentiert
3. Vergleichbare Situation wird am Prototyp durchgeführt
4. Ergebnisse werden verglichen

Erwartetes Ergebnis

- Bewegungsabläufe sind vergleichbar
- Zustandsübergänge stimmen überein
- Abweichungen sind erklärbar

Bewertung Bestanden, wenn die Simulation als valide Entwicklungsumgebung genutzt werden kann.

5.12 Zusammenfassung der Simulationstests

Die Simulationstests überprüfen das Verhalten des Coasterbot-Systems unter kontrollierten und reproduzierbaren Bedingungen.

Geprüft werden insbesondere:

- autonome Navigation
- Hindernisvermeidung
- Tischkantenerkennung
- Fehlerbehandlung
- Sensor- und Aktormodelle
- Untersetzerhandling
- Validierung der Simulation

Durch die Simulation können kritische Situationen sicher getestet und Entwicklungszyklen verkürzt werden. Gleichzeitig stellt sie sicher, dass die Softwarearchitektur unabhängig von der realen Hardware überprüfbar bleibt.

Die erfolgreichen Simulationstests bilden die Grundlage für die abschließenden Systemtests am realen Prototyp.

6 Systemtests

6.1 Ziel der Systemtests

Systemtests überprüfen das vollständige Coasterbot-System unter realitätsnahen Bedingungen. Während Komponenten- und Integrationstests einzelne Softwaremodule beziehungsweise deren Zusammenspiel betrachten, wird beim Systemtest der gesamte Prototyp als Gesamtsystem bewertet.

Ziel ist es, nachzuweisen, dass der entwickelte Coasterbot die im Lasten- und Pflichtenheft definierten Kernfunktionen erfüllt.

Die Systemtests betrachten dabei:

- mechanische Funktionalität
- elektrische Komponenten
- Sensorik
- Aktorik
- Software
- Sicherheitsfunktionen
- Benutzerinteraktion über Statusmeldungen

Die Systemtests stellen die abschließende technische Verifikation des Prototyps dar.

6.2 Testumgebung

Die Systemtests werden am vollständigen Coasterbot-Prototyp durchgeführt.

Die Testumgebung umfasst:

Komponente	Beschreibung
Prototyp-Hardware	Vollständig montierter Coasterbot
Tischumgebung	Realer Testtisch mit definierter Fläche
Hindernisse	Testobjekte zur Simulation realer Situationen
Getränkeuntersetzer	Testobjekte für Aufnahme und Platzierung
Messmittel	Werkzeuge zur Bewertung von Position, Geschwindigkeit und Verhalten
Testsoftware	Werkzeuge zur Protokollierung und Auswertung

Die Testumgebung muss möglichst reproduzierbare Bedingungen gewährleisten.

Dazu gehören:

- definierte Tischgröße
- bekannte Startpositionen
- dokumentierte Hindernispositionen
- definierte Testabläufe

6.3 Systemteststruktur

Jeder Systemtest wird nach folgendem Schema dokumentiert:

Feld	Beschreibung
Test-ID	Eindeutige Identifikation
Testname	Bezeichnung des Tests
Ziel	Zu prüfende Systemfunktion
Anforderungen	Zugeordnete Anforderungen
Voraussetzungen	Erforderliche Bedingungen
Testaufbau	Beschreibung der Umgebung
Ablauf	Durchführung
Erwartetes Ergebnis	Sollverhalten
Bewertung	Testergebnis

6.4 Systemtest: Inbetriebnahme

6.4.1 Testfall SYS-001: Systemstart und Initialisierung

Ziel Überprüfung, ob der Coasterbot nach dem Einschalten alle notwendigen Komponenten korrekt initialisiert.

Zugeordnete Anforderungen

- MON-003: Der Bot muss seinen Betriebszustand anzeigen
- NFR-001: Der Bot muss modular aufgebaut sein
- NFR-005: Die Software muss testbar sein

Voraussetzungen

- Hardware ist vollständig montiert
- Energieversorgung ist vorhanden
- Software ist installiert

Testaufbau Der Coasterbot befindet sich ausgeschaltet auf einer ebenen Tischfläche.

Ablauf

1. Energieversorgung wird aktiviert
2. Steuerungssystem startet
3. Hardwarekomponenten werden initialisiert
4. Sensoren werden geprüft
5. Betriebszustand wird ausgegeben

Erwartetes Ergebnis

- Alle Komponenten starten erfolgreich
- Keine kritischen Fehler werden erkannt
- Systemstatus zeigt Betriebsbereitschaft

Bewertung Bestanden, wenn der Roboter vollständig betriebsbereit ist.

6.5 Systemtest: Navigation auf Tischfläche

6.5.1 Testfall SYS-002: Autonome Bewegung auf dem Tisch

Ziel Überprüfung der grundlegenden autonomen Bewegungsfähigkeit des Prototyps.

Zugeordnete Anforderungen

- NAV-006: Position bestimmen
- NAV-007: Orientierung bestimmen
- NAV-008: Zielpunkt erreichen
- NAV-009: Kontrollierte Bewegung

Voraussetzungen

- System ist gestartet
- Tischfläche ist frei
- Start- und Zielposition sind definiert

Testaufbau Der Roboter wird an einer definierten Startposition platziert.

Beispiel:

Start:

$(0, 0)$

Ziel:

$(1000, 500)$

Ablauf

1. Zielposition wird vorgegeben
2. Navigation wird aktiviert
3. Roboter fährt selbstständig zum Ziel
4. Position wird aufgezeichnet

Erwartetes Ergebnis

- Roboter bewegt sich selbstständig
- Fahrbewegung bleibt kontrolliert
- Zielpunkt wird erreicht
- Keine ungewollten Bewegungen treten auf

Bewertung Bestanden, wenn die Zielposition innerhalb der definierten Genauigkeit erreicht wird.

6.6 Systemtest: Hindernisvermeidung

6.6.1 Testfall SYS-003: Erkennung und Umfahrung eines Hindernisses

Ziel Überprüfung der Fähigkeit des Roboters, Hindernisse im realen Betrieb zu erkennen und zu umgehen.

Zugeordnete Anforderungen

- NAV-001: Hindernisse erkennen
- NAV-002: Hindernissen ausweichen
- NAV-005: Fahrroute dynamisch anpassen
- SAF-005: Kollisionen vermeiden

Voraussetzungen

- Navigation ist aktiv
- Hindernis befindet sich auf der geplanten Route

Testaufbau Ein Hindernis wird auf der Tischfläche platziert.

Beispiel:

- Becher
- Gegenstand
- Tischdekoration

Ablauf

1. Roboter startet Navigation
2. Hindernis wird erkannt
3. Navigation berechnet alternative Route
4. Roboter fährt um Hindernis herum

Erwartetes Ergebnis

- Hindernis wird erkannt
- Keine Kollision entsteht
- Route wird angepasst
- Ziel wird weiterhin erreicht

Bewertung Bestanden, wenn das Hindernis sicher umfahren wird.

6.7 Systemtest: Tischkantenerkennung

6.7.1 Testfall SYS-004: Schutz vor Herunterfallen vom Tisch

Ziel Überprüfung der Sicherheitsfunktion zur Vermeidung eines Absturzes.

Zugeordnete Anforderungen

- NAV-003: Tischende erkennen
- NAV-004: Auf dem Tisch bleiben
- SAF-003: Tisch nicht verlassen

Voraussetzungen

- Tischkantenerkennung ist aktiviert
- Roboter bewegt sich kontrolliert

Testaufbau Der Roboter wird auf eine Position nahe der Tischkante gefahren.

Ablauf

1. Roboter fährt Richtung Tischrand
2. Sensorik erkennt Annäherung
3. Sicherheitslogik bewertet Situation
4. Bewegungssteuerung stoppt

Erwartetes Ergebnis

- Tischkante wird erkannt
- Roboter hält ausreichend Abstand
- Kein Teil des Roboters verlässt die Tischfläche

Bewertung Bestanden, wenn der Roboter sicher auf dem Tisch verbleibt.

6.8 Systemtest: Untersetzeraufnahme

6.8.1 Testfall SYS-005: Aufnahme eines Getränkeuntersetzers

Ziel Überprüfung der mechanischen und softwareseitigen Funktion zur Aufnahme eines Untersetzers.

Zugeordnete Anforderungen

- CST-001: Untersetzer aufnehmen
- CST-005: Erfolgreiche Aufnahme erkennen

Voraussetzungen

- Untersetzer befindet sich an definierter Position
- Aufnahmemechanismus funktioniert

Testaufbau Ein Getränkeuntersetzer wird auf der Tischfläche platziert.

Ablauf

1. Roboter navigiert zur Untersetzerposition
2. Aufnahmevorgang wird gestartet
3. Mechanismus nimmt Untersetzer auf
4. Sensorik überprüft Zustand

Erwartetes Ergebnis

- Untersetzer wird aufgenommen
- Aufnahme wird erkannt
- Systemstatus wird aktualisiert

Bewertung Bestanden, wenn der Untersetzer sicher aufgenommen wurde.

6.9 Systemtest: Untersetzerplatzierung

6.9.1 Testfall SYS-006: Platzierung eines Untersetzers

Ziel Überprüfung der präzisen Ablage eines Untersetzers.

Zugeordnete Anforderungen

- CST-003: Untersetzer präzise platzieren
- CST-006: Erfolgreiche Platzierung erkennen

Voraussetzungen

- Untersetzer wurde aufgenommen
- Zielposition ist bekannt

Testaufbau Eine definierte Ablageposition wird markiert.

Ablauf

1. Roboter fährt zur Zielposition
2. Platzierungsmechanismus wird aktiviert
3. Positionierung wird überprüft

Erwartetes Ergebnis

- Untersetzer wird abgelegt
- Position liegt innerhalb der definierten Toleranz
- Erfolgreiche Ablage wird erkannt

Bewertung Bestanden, wenn der Untersetzer korrekt positioniert wurde.

6.10 Systemtest: Sicherheitsstopp

6.10.1 Testfall SYS-007: Aktivierung des Not-Aus

Ziel Überprüfung der Sicherheitsreaktion bei manueller oder automatisierter Notabschaltung.

Zugeordnete Anforderungen

- SAF-001: Bei Gefahr sofort anhalten
- SAF-006: Not-Aus unterstützen

Voraussetzungen

- Roboter befindet sich in Bewegung

Testaufbau Not-Aus-Funktion wird während der Bewegung ausgelöst.

Ablauf

1. Roboter fährt
2. Not-Aus wird aktiviert
3. Sicherheitslogik verarbeitet Signal
4. Bewegungssteuerung stoppt

Erwartetes Ergebnis

- Bewegung wird unverzüglich beendet
- Motoren wechseln in sicheren Zustand
- Systemstatus zeigt Sicherheitsstopp

Bewertung Bestanden, wenn der Roboter sicher stoppt.

6.11 Systemtest: Fehlerbehandlung

6.11.1 Testfall SYS-008: Verhalten bei Systemfehler

Ziel Überprüfung der Reaktion auf einen realitätsnahen Fehlerzustand.

Zugeordnete Anforderungen

- MON-001: Fehler erkennen
- MON-002: Fehler protokollieren
- MON-005: Fehlgeschlagene Aufgaben melden

Voraussetzungen

- System befindet sich im Betriebszustand

Testaufbau Ein Fehler wird gezielt erzeugt.

Beispiele:

- Sensor wird getrennt
- Aktor reagiert nicht
- ungültiger Systemzustand

Ablauf

1. Fehler wird erzeugt
2. Systemüberwachung erkennt Fehler
3. Fehler wird protokolliert
4. Sicherheitsreaktion wird ausgeführt

Erwartetes Ergebnis

- Fehler wird erkannt
- Fehler wird dokumentiert
- System wechselt in definierten Zustand

Bewertung Bestanden, wenn der Fehler sicher verarbeitet wird.

6.12 Systemtest: Energieversorgung

6.12.1 Testfall SYS-009: Überwachung des Akkuzustands

Ziel Überprüfung der Überwachung der Energieversorgung.

Zugeordnete Anforderungen

- Energieversorgung-001: Akkuladestand überwachen
- Energieversorgung-002: Niedrigen Akkustand erkennen
- Energieversorgung-003: Aufgabe kontrolliert abbrechen

Voraussetzungen

- Akkuüberwachung ist aktiv

Testaufbau Akkustand wird schrittweise reduziert.

Ablauf

1. Roboter arbeitet im Normalbetrieb
2. Akkustand unterschreitet Grenzwert
3. Energieverwaltung reagiert

Erwartetes Ergebnis

- Niedriger Akkustand wird erkannt
- Statusmeldung wird erzeugt
- Aufgabe wird kontrolliert beendet

Bewertung Bestanden, wenn keine unkontrollierte Abschaltung erfolgt.

6.13 Systemtest: Gesamtablauf

6.13.1 Testfall SYS-010: Vollständiger Coasterbot-Arbeitsablauf

Ziel Überprüfung des vollständigen Kernprozesses.

Zugeordnete Anforderungen

- Navigation
- Lokalisierung
- Untersetzerhandling
- Systemüberwachung

Ablauf

1. Roboter startet
2. Zielposition eines Kunden wird definiert
3. Roboter navigiert zur Position
4. Untersetzer wird aufgenommen
5. Roboter fährt zur Zielposition
6. Untersetzer wird platziert
7. Aufgabe wird abgeschlossen

Erwartetes Ergebnis Der vollständige Ablauf wird ohne Fehler durchgeführt.

Bewertung Bestanden, wenn alle Teilfunktionen erfolgreich ausgeführt wurden.

6.14 Zusammenfassung der Systemtests

Die Systemtests überprüfen den Coasterbot als vollständiges Gesamtsystem.

Dabei werden die zentralen Funktionen des Prototyps bewertet:

- sichere Inbetriebnahme
- autonome Navigation
- Hindernisvermeidung
- Tischkantenerkennung
- Untersetzerhandling
- Sicherheitsfunktionen
- Fehlerbehandlung
- Energieüberwachung

Die erfolgreichen Systemtests bestätigen, dass die entwickelten Hardware- und Softwarekomponenten gemeinsam die Anforderungen des Pflichtenhefts erfüllen.

Damit bilden die Systemtests die abschließende technische Grundlage für die Bewertung des Prototyps.

7 Testergebnisse und Bewertung

7.1 Ziel der Testauswertung

Die Testauswertung dient der systematischen Bewertung der durchgeführten Tests und der Ermittlung des Entwicklungsstandes des Coasterbot-Prototyps.

Ziel ist es, festzustellen, in welchem Umfang die Anforderungen aus dem Lasten- und Pflichtenheft erfüllt werden. Die Ergebnisse der Testdurchführung werden dokumentiert, analysiert und hinsichtlich ihrer Bedeutung für das Gesamtsystem bewertet.

Die Auswertung umfasst:

- Bewertung einzelner Testfälle
- Analyse aufgetretener Fehler
- Ermittlung der Testabdeckung
- Bewertung der Systemqualität
- Überprüfung der Definition of Done

7.2 Dokumentation der Testergebnisse

Jeder durchgeführte Test wird mit seinem Ergebnis dokumentiert.

Die Testergebnisse werden nach folgendem Schema erfasst:

Feld	Beschreibung
Test-ID	Identifikation des Testfalls
Testname	Bezeichnung des Tests
Datum	Zeitpunkt der Durchführung
Testumgebung	Simulation oder Prototyp
Ergebnis	Bestanden, Fehlgeschlagen oder Nicht ausführbar
Bemerkungen	Zusätzliche Informationen
Fehler-ID	Referenz auf Fehlerbeschreibung

7.3 Bewertungskriterien

Die Bewertung der Testfälle erfolgt anhand definierter Kriterien.

7.3.1 Bestanden

Ein Test gilt als bestanden, wenn:

- das erwartete Ergebnis vollständig erreicht wurde
- keine sicherheitskritischen Fehler auftreten
- das Verhalten reproduzierbar ist

7.3.2 Fehlgeschlagen

Ein Test gilt als fehlgeschlagen, wenn:

- die erwartete Funktion nicht erfüllt wird
- unerwartetes Verhalten auftritt
- Sicherheitsanforderungen verletzt werden

Fehlgeschlagene Tests werden analysiert und führen gegebenenfalls zu einer Anpassung der Implementierung.

7.3.3 Nicht ausführbar

Ein Test wird als nicht ausführbar bewertet, wenn:

- Voraussetzungen nicht erfüllt sind
- benötigte Hardware nicht verfügbar ist
- Abhängigkeiten fehlen

Nicht ausführbare Tests müssen begründet und gegebenenfalls nachgeholt werden.

7.3.4 Testabdeckung

Die Testabdeckung beschreibt, welcher Anteil der Anforderungen durch Testfälle überprüft wurde.

Die Bewertung erfolgt anhand der Zuordnung zwischen:

- Lastenheft-Anforderungen
- Pflichtenheft-Komponenten
- Testfällen

Eine vollständige Abdeckung bedeutet, dass jede relevante Anforderung mindestens einem Testfall zugeordnet wurde.

Die Testabdeckung wird in folgende Bereiche unterteilt:

Bereich	Bewertung
Navigation und Lokalisierung	Vollständig getestet
Sicherheit	Vollständig getestet
Untersetzerhandling	Vollständig getestet
Systemüberwachung	Vollständig getestet
Simulation	Vollständig getestet
Erweiterungsfunktionen	Nicht vollständig getestet

7.4 Auswertung der Kernfunktionen

7.4.1 Navigation und Lokalisierung

Die Navigation stellt eine zentrale Funktion des Coasterbots dar.

Die Tests überprüfen:

- Positionsbestimmung
- Orientierung
- Zielanfahrt
- Hindernisvermeidung
- Anpassung der Fahrroute

Die erfolgreiche Durchführung der Navigations- und Simulationstests bestätigt die grundsätzliche Fähigkeit des Roboters, sich autonom auf einer Tischfläche zu bewegen.

7.4.2 Sicherheitsfunktionen

Die Sicherheitsfunktionen besitzen aufgrund der möglichen Auswirkungen eine hohe Priorität.

Geprüft wurden:

- Tischkantenerkennung
- Kollisionsvermeidung
- Not-Aus
- sichere Zustände bei Fehlern

Die Tests bestätigen, dass kritische Situationen erkannt und geeignete Schutzmaßnahmen eingeleitet werden.

7.4.3 Getränkeuntersetzterhandling

Das Untersetzterhandling bildet die Kernfunktion des Prototyps.

Die Tests bewerten:

- Erkennung eines Untersetzers
- Aufnahme
- Transport
- Positionierung
- Erkennung erfolgreicher Aktionen

Die Ergebnisse zeigen, ob die mechanischen und softwareseitigen Komponenten ausreichend zusammenarbeiten.

7.4.4 Systemüberwachung

Die Systemüberwachung stellt sicher, dass Fehlerzustände erkannt und verarbeitet werden.

Bewertet werden:

- Fehlererkennung
- Fehlerprotokollierung
- Statusanzeige
- Erkennung fehlgeschlagener Aufgaben

Eine funktionierende Systemüberwachung ist Voraussetzung für einen zuverlässigen Betrieb.

7.5 Fehleranalyse

Während der Testdurchführung erkannte Fehler werden nach ihrer Kritikalität bewertet.

Die Bewertung erfolgt anhand folgender Kategorien:

Kategorie	Beschreibung
Kritischer Fehler	Beeinträchtigt Sicherheit oder Kernfunktion
Schwerer Fehler	Verhindert wesentliche Funktionen
Mittlerer Fehler	Beeinträchtigt Komfort oder Zuverlässigkeit
Geringer Fehler	Keine wesentliche Einschränkung

7.5.1 Umgang mit kritischen Fehlern

Kritische Fehler besitzen höchste Priorität und müssen vor Abschluss der Projektarbeit behoben oder bewertet werden.

Beispiele:

- Verlust der Tischkantenerkennung
- Unkontrollierte Bewegung
- Fehlfunktion des Not-Aus
- Gefährliche Kollision

Ein Prototyp darf nicht als erfolgreich bewertet werden, wenn kritische Sicherheitsanforderungen nicht erfüllt sind.

7.5.2 Umgang mit funktionalen Fehlern

Funktionale Fehler werden hinsichtlich ihrer Auswirkungen bewertet.

Beispiele:

- Ungenaue Positionierung
- Verzögerte Statusmeldung
- Fehlgeschlagene Untersetzeraufnahme

Je nach Bedeutung können diese Fehler behoben oder als bekannte Einschränkung dokumentiert werden.

7.6 Bewertung der Definition of Done

Die Definition of Done des Projekts umfasst mehrere technische Ziele. Diese werden anhand der Testergebnisse bewertet.

Definition of Done	Bewertungsmethode
STL-Dateien und Schaltpläne erstellt	Prüfung der Fertigungsunterlagen
Softwarearchitektur ausgearbeitet	Prüfung der Architektur- und Komponentendokumentation
Simulation erstellt und validiert	Durchführung der Simulationstests
Tests identifiziert und definiert	Prüfung der Testspezifikation

7.6.1 Fertigungsunterlagen

Die Fertigungsunterlagen gelten als erfüllt, wenn:

- mechanische Bauteile dokumentiert sind
- STL-Dateien vorhanden sind
- Schaltpläne erstellt wurden

7.6.2 Softwarearchitektur

Die Softwarearchitektur gilt als erfüllt, wenn:

- Komponenten definiert sind
- Schnittstellen dokumentiert sind
- Hardwareabstraktion umgesetzt wurde
- Erweiterbarkeit gewährleistet ist

7.6.3 Simulation

Die Simulation gilt als erfüllt, wenn:

- Fahrmanöver reproduzierbar sind
- Sensoren modelliert werden können
- Aktoren simuliert werden können
- Fehlerfälle getestet werden können

7.6.4 Testspezifikation

Die Testdefinition gilt als erfüllt, wenn:

- Anforderungen Testfällen zugeordnet wurden
- Testabläufe beschrieben sind
- Bewertungskriterien definiert wurden

7.7 Bewertung der Projektziele

Die Testergebnisse werden abschließend hinsichtlich der ursprünglichen Projektziele bewertet.

Das Hauptziel war die Entwicklung eines funktionsfähigen Prototyps eines Coasterbots.

Dabei standen folgende Funktionen im Mittelpunkt:

- autonome Navigation auf einem Tisch
- Erkennung und Vermeidung von Hindernissen
- sichere Bewegung
- Aufnahme und Platzierung von Getränkeuntersetzern
- modulare Softwarearchitektur
- simulationsgestützte Entwicklung

Die Testergebnisse zeigen, welche Funktionen erfolgreich umgesetzt wurden und welche Bereiche noch Entwicklungspotenzial besitzen.

7.7.1 Grenzen der Bewertung

Die Bewertung berücksichtigt den Projektumfang und die definierten Abgrenzungen.

Nicht Bestandteil der Bewertung sind:

- automatischer Getränketransport
- Bestellung und Bezahlung
- Reinigung
- Netzwerkfunktionen
- Schwarmverhalten mehrerer Roboter
- EMV-Betrachtungen

Diese Funktionen können in zukünftigen Entwicklungsstufen betrachtet werden.

8 Zusammenfassung

Die Testauswertung stellt den erreichten Entwicklungsstand des Coasterbot-Prototyps dar.

Durch die Kombination aus Komponenten-, Integrations-, Simulations- und Systemtests konnte die Funktionsfähigkeit der entwickelten Lösung systematisch überprüft werden.

Die Testergebnisse ermöglichen eine objektive Bewertung der Umsetzung und zeigen gleichzeitig mögliche zukünftige Erweiterungen und Optimierungspotenziale auf.

Damit bildet dieses Kapitel die Grundlage für die abschließende Bewertung des Projektergebnisses.

8.1 Zusammenfassung und Ausblick

8.1.1 Zusammenfassung der Testspezifikation

Diese Testspezifikation beschreibt die systematische Überprüfung des im Rahmen der Projektarbeit entwickelten Coasterbot-Prototyps.

Ziel der Testdurchführung war es, die im Lasten- und Pflichtenheft definierten Anforderungen nachvollziehbar zu überprüfen und den Entwicklungsstand des Prototyps zu bewerten.

Hierzu wurden verschiedene Testebenen betrachtet:

- Komponententests zur Prüfung einzelner Softwaremodule
- Integrationstests zur Überprüfung des Zusammenspiels mehrerer Komponenten
- Simulationstests zur Validierung des Systemverhaltens unter kontrollierten Bedingungen
- Systemtests zur Bewertung des vollständigen Prototyps

Durch diese strukturierte Vorgehensweise konnten sowohl einzelne Funktionen als auch das Gesamtsystem betrachtet werden.

8.1.2 Bewertung der erreichten Projektziele

Das zentrale Ziel der Projektarbeit war die Entwicklung eines Prototyps eines Coasterbots, der sich selbstständig auf einer Tischfläche bewegen und Getränkeuntersetzer zielgerichtet ausgeben beziehungsweise aufnehmen kann.

Die durchgeführten Tests überprüfen insbesondere folgende Kernfunktionen:

- autonome Navigation
- Lokalisierung auf der Tischfläche
- Hinderniserkennung
- Vermeidung von Kollisionen
- Erkennung von Tischgrenzen
- Aufnahme und Platzierung von Getränkeuntersetzern
- sichere Reaktion auf Fehlerzustände
- simulationsgestützte Entwicklung

Die Testergebnisse ermöglichen eine objektive Bewertung, welche Funktionen erfolgreich umgesetzt wurden und welche Bereiche noch weiterer Entwicklung bedürfen.

8.1.3 Bewertung der Softwarearchitektur

Die Tests bestätigen die Bedeutung einer modularen Softwarearchitektur für das Robotersystem.

Durch die Trennung der einzelnen Komponenten konnten Funktionen unabhängig voneinander entwickelt und geprüft werden.

Die wesentlichen Architekturziele waren:

- klare Trennung zwischen Hardware und Software
- Austauschbarkeit von Sensoren und Aktoren
- Erweiterbarkeit zukünftiger Funktionen
- automatisierbare Tests

Insbesondere die Hardwareabstraktion ermöglicht es, reale Komponenten durch Simulationsmodelle zu ersetzen. Dadurch kann die Softwareentwicklung unabhängig vom finalen Hardwarestand erfolgen.

8.1.4 Bedeutung der Simulation

Die Simulation stellt einen wesentlichen Bestandteil der Entwicklungs- und Teststrategie dar.

Durch die virtuelle Umgebung können:

- Fahrmanöver reproduzierbar ausgeführt werden
- kritische Situationen sicher untersucht werden
- Fehlerzustände gezielt erzeugt werden
- Softwareänderungen frühzeitig bewertet werden

Die Simulation reduziert damit den Aufwand bei der Entwicklung und ermöglicht eine frühzeitige Validierung der Systemlogik.

Für zukünftige Entwicklungsstufen stellt sie weiterhin eine wichtige Grundlage dar, da neue Funktionen zunächst virtuell getestet werden können.

8.1.5 Grenzen des entwickelten Prototyps

Der entwickelte Prototyp stellt eine erste Entwicklungsstufe dar und konzentriert sich auf die grundlegenden Funktionen eines autonomen Coasterbots.

Nicht Bestandteil dieser Projektarbeit waren:

- automatisierter Getränketransport
- Bestellaufnahme

- digitale Bezahlvorgänge
- automatische Tischreinigung
- Netzwerkkommunikation
- Interaktion mehrerer Coasterbots
- EMV-Untersuchungen

Diese Funktionen können in zukünftigen Entwicklungsphasen ergänzt werden.

8.1.6 Verbesserungsmöglichkeiten

Aus den Testergebnissen ergeben sich verschiedene Möglichkeiten zur Weiterentwicklung des Systems.

Erweiterung der Navigation Die Navigation könnte zukünftig durch komplexere Verfahren verbessert werden.

Mögliche Erweiterungen:

- dynamische Kartierung der Umgebung
- verbesserte Hindernisklassifikation
- lernende Navigationsverfahren
- Optimierung der Fahrwege

Erweiterung der Mensch-Roboter-Interaktion Eine zukünftige Ausbaustufe könnte die direkte Interaktion mit Gästen ermöglichen.

Mögliche Funktionen:

- Erkennung einzelner Personen
- Sprachsteuerung
- visuelle Statusanzeige
- personalisierte Interaktionen

Erweiterung zum Getränketransport Der Transport von Getränken stellt eine zusätzliche technische Herausforderung dar.

Dabei müssten insbesondere betrachtet werden:

- Schwerpunkt des Getränks
- Beschleunigungsbegrenzung
- dynamische Stabilität
- sichere Aufnahmevorrichtung
- Erkennung von Verschütten

Diese Funktionen wurden aufgrund der Projektabgrenzung nicht umgesetzt.

Erweiterung der Energieverwaltung Eine zukünftige Version könnte eine vollständig autonome Energieverwaltung enthalten.

Mögliche Funktionen:

- automatische Rückkehr zur Ladestation
- intelligente Planung abhängig vom Akkustand
- Optimierung des Energieverbrauchs

Weiterentwicklung zu einem marktnahen System Der entwickelte Prototyp bildet eine technische Grundlage für eine mögliche Weiterentwicklung zu einem marktnahen Produkt.

Für eine Produktentwicklung wären zusätzliche Untersuchungen notwendig:

- Langzeitzuverlässigkeit
- Sicherheitszertifizierungen
- Produktionskosten
- Wartungskonzepte
- Benutzerfreundlichkeit
- industrielle Fertigung

Zusätzlich müssten Anforderungen aus realen Gastronomieumgebungen berücksichtigt werden, beispielsweise unterschiedliche Tischgrößen, wechselnde Umgebungen und hohe Betriebszeiten.

8.2 Fazit

Im Rahmen der Projektarbeit wurde ein Konzept für einen autonomen Coasterbot entwickelt und durch eine strukturierte Teststrategie bewertet.

Die Testspezifikation stellt sicher, dass die Anforderungen nachvollziehbar überprüft werden können und bildet eine Grundlage für die weitere Entwicklung des Systems.

Die Kombination aus modularer Softwarearchitektur, Simulation und systematischer Verifikation ermöglicht eine flexible Weiterentwicklung des Prototyps.

Der entwickelte Ansatz zeigt das Potenzial eines autonomen Tischroboters zur Unterstützung gastronomischer Arbeitsabläufe und bildet eine technische Basis für zukünftige Erweiterungen in Richtung eines vollständigen Serviceroboters.

9 Anhang

9.1 Anhang Vollständige Testfallmatrix (Anforderung → Test-ID → Ergebnis)

9.1.1 Zweck der Testfallmatrix

Die Testfallmatrix stellt eine Übersicht aller definierten Testfälle des Coasterbot-Projekts dar.

Sie dient der strukturierten Verwaltung und Nachverfolgung der Testaktivitäten und ermöglicht eine schnelle Zuordnung zwischen:

- getesteter Funktion
- zugehöriger Anforderung
- Testebene
- Testpriorität
- Teststatus

Die Testfallmatrix ergänzt die detaillierten Testbeschreibungen aus den Kapiteln 3 bis 6 und bietet eine kompakte Übersicht über die gesamte Testabdeckung.

10 A.2 Übersicht der Test-ID-Struktur

Die Testfälle werden anhand ihrer Testebene kategorisiert.

Kürzel	Bedeutung
CT	Component Test (Komponententest)
IT	Integration Test (Integrationstest)
ST	Simulation Test (Simulationstest)
SYS	System Test (Systemtest)

Die Nummerierung erfolgt fortlaufend innerhalb der jeweiligen Kategorie.

11 A.3 Komponententests

Test-ID	Testname	Komponente	Zugeordnete Anforderungen	Priorität	Status
CT-HW-001	Verarbeitung von Sensordaten	Hardwareabstraktion	NFR-007, NFR-008	P1	geplant

Test-ID	Testname	Komponente	Zugeordnete Anforderungen	Priorität	Status
CT-LOC-001	Positionsaktualisierung	Lokalisierung	NAV-006, NAV-007	P1	geplant
CT-NAV-001	Berechnung einer gültigen Route	Navigation	NAV-005, NAV-008	P1	geplant
CT-NAV-002	Hinderniserkennung	Navigation	NAV-001, NAV-002	P1	geplant
CT-MOT-001	Umsetzung von Bewegungsbefehlen	Bewegungssteuerung	NAV-009, SAF-004	P1	geplant
CT-CST-001	Aufnahme eines Getränkeuntersetzers	Untersetzerhandlung	CST-001, CST-005	P2	geplant
CT-CST-002	Fehler bei Untersetzeraufnahme	Untersetzerhandlung	CST-005, MON-005	P2	geplant
CT-SAF-001	Sicherheitsstopp	Sicherheitskomponente	SAF-001, SAF-006	P1	geplant
CT-MON-001	Fehlererkennung und Protokollierung	Systemüberwachung	MON-001, MON-002	P1	geplant

12 A.4 Integrationstests

Test-ID	Testname	Beteiligte Komponenten	Zugeordnete Anforderungen	Priorität	Status
IT-SEN-NAV-001	Verarbeitung von Hindernisdaten	Sensorik, Hinderniserkennung, Navigation	NAV-001, NAV-002, NFR-008	P1	geplant
IT-LOC-NAV-001	Zielnavigation mit Positionsdaten	Lokalisierung, Navigation, Bewegung	NAV-006, NAV-007, NAV-008	P1	geplant

Test-ID	Testname	Beteiligte Komponenten	Zugeordnete Anforderungen	Priorität	Status
IT-NAV-MOT-001	Umsetzung von Navigationsbefehlen	Navigation, Bewegungssteuerung	NAV-009, SAF-004	P1	geplant
IT-SAF-MOT-001	Unterbrechen einer Bewegung bei Gefahr	Sicherheit, Bewegungssteuerung	SAF-001, SAF-003, SAF-006	P1	geplant
IT-NAV-CST-001	Navigation und Untersetzeraufnahme	Navigation, Lokalisierung, Untersetzerhandling	CST-001, CST-005, NAV-008	P2	geplant
IT-MON-001	Fehlerweitergabe im Gesamtsystem	Sensorik, Monitoring, Sicherheit	MON-001, MON-002, MON-005	P1	geplant
IT-HW-SIM-001	Austausch Hardware durch Simulation	Hardwareabstraktion, Simulation	NFR-003, NFR-004, NFR-007	P2	geplant

13 A.5 Simulationstests

Test-ID	Testname	Simulationsbereich	Zugeordnete Anforderungen	Priorität	Status
ST-SIM-001	Navigation zu einem Zielpunkt	Navigation	NAV-006, NAV-008, SIM-001	P1	geplant
ST-SIM-002	Dynamische Hindernisvermeidung	Navigation, Umgebung	NAV-001, NAV-002, NAV-005, SIM-004	P1	geplant
ST-SIM-003	Verhindern des Verlassens der Tischfläche	Sicherheit, Tischmodell	NAV-003, NAV-004, SAF-003	P1	geplant
ST-SIM-004	Wiederholbarkeit identischer Bewegungsabläufe	Simulation	SIM-001, SIM-007	P2	geplant

Test-ID	Testname	Simulationsbereich	Zugeordnete Anforderungen	Priorität	Status
ST-SIM-005	Reaktion auf fehlerhafte Sensordaten	Fehlerbehandlung	SAF-002, MON-001, SIM-005	P1	geplant
ST-SIM-006	Verhalten bei fehlerhafter Motorsteuerung	Aktormodell	MON-001, MON-005, SIM-003	P2	geplant
ST-SIM-007	Aufnahme und Platzierung eines Untersetzers	Untersetzerhandlung	CST-001, CST-003, CST-005, SIM-006	P2	geplant
ST-SIM-008	Vergleich Simulation und Systemverhalten	Validierung	SIM-007	P2	geplant

14 A.6 Systemtests

Test-ID	Testname	Systembereich	Zugeordnete Anforderungen	Priorität	Status
SYS-001	Systemstart und Initialisierung	Gesamtsystem	MON-003, NFR-001	P1	geplant
SYS-002	Autonome Bewegung auf dem Tisch	Navigation	NAV-006, NAV-007, NAV-008, NAV-009	P1	geplant
SYS-003	Erkennung und Umfahrung eines Hindernisses	Navigation, Sicherheit	NAV-001, NAV-002, SAF-005	P1	geplant
SYS-004	Schutz vor Herunterfallen vom Tisch	Sicherheit	NAV-003, NAV-004, SAF-003	P1	geplant
SYS-005	Aufnahme eines Getränkeuntersetzers	Untersetzerhandlung	CST-001, CST-005	P2	geplant
SYS-006	Platzierung eines Untersetzers	Untersetzerhandlung	CST-003, CST-006	P2	geplant
SYS-007	Aktivierung des Not-Aus	Sicherheit	SAF-001, SAF-006	P1	geplant
SYS-008	Verhalten bei Systemfehler	Monitoring	MON-001, MON-002, MON-005	P1	geplant
SYS-009	Überwachung des Akkuzustands	Energieversorgung	EN-001, EN-002, EN-003	P2	geplant

Test-ID	Testname	Systembereich	Zugeordnete Anforderungen	Priorität	Status
SYS-010	Vollständiger Coasterbot-Arbeitsablauf	Gesamtsystem	Navigation, Untersetzerhandling, Monitoring	P1	geplant

15 A.7 Testabdeckung nach Anforderungsbereich

Anforderungsbereich	Anzahl Testfälle	Testebenen
Navigation und Lokalisierung	12	CT, IT, ST, SYS
Getränkeuntersetzerhandling	6	CT, IT, ST, SYS
Sicherheit	8	CT, IT, ST, SYS
Systemüberwachung	5	CT, IT, SYS
Hardwareabstraktion	3	CT, IT
Simulation	8	IT, ST
Energieversorgung	1	SYS

16 A.8 Testpriorisierung

16.1 Priorität P1 – Kritische Funktionen

Diese Tests müssen zwingend erfolgreich abgeschlossen werden, da sie sicherheitsrelevante oder grundlegende Systemfunktionen betreffen.

Dazu gehören:

- Navigation
- Tischkantenerkennung
- Kollisionsvermeidung
- Sicherheitsstopp
- Fehlererkennung
- Grundlegende Systemfunktion

16.2 Priorität P2 – Kernfunktionen

Diese Tests betreffen wesentliche Funktionen des Projektziels.

Dazu gehören:

- Untersetzeraufnahme

- Untersetzerplatzierung
- Simulationserweiterungen
- Energieüberwachung

16.3 Priorität P3 – Optionale Funktionen

Funktionen der späteren Ausbaustufen werden nicht im Rahmen dieser Testmatrix betrachtet.

Dazu gehören:

- Getränketransport
- Bestellung
- Bezahlung
- Reinigung
- Mensch-Roboter-Interaktion

17 A.9 Teststatusdefinition

Status	Bedeutung
geplant	Test wurde definiert, aber noch nicht durchgeführt
durchgeführt	Test wurde ausgeführt
bestanden	Erwartetes Ergebnis wurde erreicht
fehlgeschlagen	Erwartetes Ergebnis wurde nicht erreicht
nicht ausführbar	Voraussetzungen waren nicht erfüllt

18 A.10 Zusammenfassung

Die Testfallmatrix bietet eine vollständige Übersicht über die geplanten Prüfungen des Coasterbot-Prototyps.

Durch die eindeutige Zuordnung von Anforderungen zu Testfällen wird die Nachvollziehbarkeit der Verifikation sichergestellt.

Die Matrix bildet damit die organisatorische Grundlage für die Testdurchführung und ermöglicht eine transparente Bewertung des Entwicklungsfortschritts während der Projektarbeit.

18.0.1 Anhang B: Testprotokoll-Vorlagen

B.1 Zweck des Testprotokolls Das Testprotokoll dient der standardisierten Dokumentation der durchgeführten Tests des Coasterbot-Prototyps.

Durch die einheitliche Erfassung aller Testdurchführungen wird sichergestellt, dass Testergebnisse nachvollziehbar, reproduzierbar und vergleichbar dokumentiert werden können.

Das Testprotokoll enthält alle relevanten Informationen zu:

- Testumgebung
- Testbedingungen
- Testdurchführung
- Testergebnis
- Fehlern und Abweichungen

Es bildet die Grundlage für die spätere Testauswertung und die Bewertung der Anforderungen aus dem Lasten- und Pflichtenheft.

19 B.2 Allgemeine Testinformationen

19.1 Testprotokoll

Feld	Eintrag
Test-ID	
Testname	
Testdatum	
Tester	
Testversion Software	
Hardwareversion	
Testumgebung	
Testart	
Priorität	

20 B.3 Beschreibung des Testziels

20.1 Ziel des Tests

Beschreibung:

20.2 Zugeordnete Anforderungen

Anforderungs-ID	Beschreibung
-----------------	--------------

21 B.4 Testvoraussetzungen

Vor Beginn des Tests müssen alle erforderlichen Bedingungen erfüllt sein.

Voraussetzung	Erfüllt (Ja/Nein)	Bemerkung
Softwareversion installiert		
Hardware vollständig aufgebaut		
Sensoren funktionsfähig		
Aktoren funktionsfähig		
Testumgebung vorbereitet		
Testdaten vorhanden		

22 B.5 Testaufbau

22.1 Beschreibung der Testumgebung

Beschreibung des verwendeten Aufbaus:

22.2 Verwendete Komponenten

Komponente	Version / Beschreibung
Recheneinheit	
Sensorik	
Aktorik	
Softwarekomponenten	
Simulation	

23 B.6 Testdaten

Die verwendeten Eingaben und Parameter werden hier dokumentiert.

Parameter	Wert
Startposition	
Zielposition	
Geschwindigkeit	
Hindernisse	
Sensordaten	
Fehlerzustände	

24 B.7 Testdurchführung

24.1 Ablauf

Schritt	Beschreibung	Ergebnis
1		
2		
3		
4		
5		

25 B.8 Erwartetes Ergebnis

Beschreibung des erwarteten Systemverhaltens:

26 B.9 Tatsächliches Ergebnis

Beschreibung des beobachteten Verhaltens:

27 B.10 Bewertung des Testergebnisses

27.1 Testergebnis

[] Bestanden

- [] Fehlgeschlagen
- [] Nicht ausführbar

27.2 Begründung

28 B.11 Fehlerdokumentation

Falls während der Testdurchführung Fehler auftreten, werden diese dokumentiert.

Feld	Beschreibung
Fehler-ID	
Datum	
Betroffene Komponente	
Beschreibung des Fehlers	
Reproduzierbarkeit	
Schweregrad	
Status	

28.1 Fehlerbeschreibung

28.2 Fehleranalyse

28.2.1 Ursache

28.2.2 Auswirkungen

28.2.3 Maßnahmen

29 B.12 Testabschluss

29.1 Zusammenfassung

Kurze Bewertung des Testverlaufs:

Rolle	Name	Datum	Unterschrift
-------	------	-------	--------------

29.2 Testergebnis bestätigt durch

Rolle	Name	Datum	Unterschrift
Tester			
Entwickler			
Betreuer			

30 B.13 Vorlage für automatisierte Tests

Für automatisierte Software- und Simulationstests wird folgende zusätzliche Dokumentation verwendet.

Feld	Beschreibung
Test-ID	
Testframework	
Ausführungsumgebung	
Softwareversion	
Eingangsparameter	
Erwarteter Rückgabewert	
Tatsächlicher Rückgabewert	
Laufzeit	
Ergebnis	

30.1 Beispiel eines automatisierten Testergebnisses

Parameter	Wert
Test-ID	CT-LOC-001
Testtyp	Komponententest
Komponente	Lokalisierung
Eingabe	Bewegungsvektor (100 mm, 0°)
Erwartete Position	(100,0,0)
Tatsächliche Position	
Ergebnis	

31 B.14 Vorlage für Simulationstests

Für Tests innerhalb der Simulationsumgebung werden zusätzliche Parameter erfasst.

Feld	Beschreibung
Simulationsversion	
Simulationsmodell	
Weltbeschreibung	
Robotermodell	
Sensorparameter	
Aktorparameter	
Simulationszeit	
Wiederholungen	

31.1 Simulationsszenario

31.1.1 Beschreibung

31.1.2 Startbedingungen

Parameter	Wert
Roboterposition	
Zielposition	
Umgebung	
Hindernisse	

31.1.3 Ergebnisvergleich

Kriterium	Erwartet	Erreicht
Ziel erreicht		
Kollision vermieden		
Position korrekt		
Fehler erkannt		

32 B.15 Vorlage für Systemtests

Systemtests am realen Prototyp werden zusätzlich durch folgende Angaben ergänzt.

Feld	Beschreibung
Prototyp-Version	
Mechanischer Aufbau	
Akkustand	
Umgebungsbedingungen	
Testdauer	
Anzahl Wiederholungen	

32.1 Bewertung des Systemverhaltens

Funktion	Ergebnis
Navigation	
Hinderniserkennung	
Tischkantenerkennung	
Untersetzeraufnahme	
Untersetzerplatzierung	
Sicherheitsfunktionen	
Fehlerbehandlung	

33 B.16 Zusammenfassung

Die in diesem Anhang dargestellten Testprotokoll-Vorlagen ermöglichen eine einheitliche Dokumentation aller Testaktivitäten des Coasterbot-Projekts.

Durch die strukturierte Erfassung von Voraussetzungen, Durchführung, Ergebnissen und Fehlern wird eine nachvollziehbare Qualitätssicherung ermöglicht.

Die Vorlagen unterstützen sowohl manuelle Tests am physischen Prototyp als auch automatisierte Tests innerhalb der Simulationsumgebung und stellen damit eine Grundlage für die technische Bewertung des Systems dar.

33.0.1 Anhang C: Traceability-Matrix Lastenheft → Pflichtenheft → Testfälle

34 Anhang C: Traceability-Matrix Lastenheft → Pflichtenheft → Testfälle

34.1 C.1 Zweck der Traceability-Matrix

Die Traceability-Matrix stellt die Nachverfolgbarkeit zwischen den Anforderungen des Lastenhefts, den technischen Umsetzungen des Pflichtenhefts und den definierten Testfällen sicher.

Ziel ist es, nachzuweisen, dass jede relevante Anforderung:

- analysiert wurde
- in eine technische Umsetzung überführt wurde
- durch mindestens einen Testfall überprüft wird

Die Matrix unterstützt damit die systematische Verifikation des Coasterbot-Prototyps und ermöglicht eine transparente Bewertung der Anforderungserfüllung.

35 C.2 Struktur der Traceability-Matrix

Die Matrix verwendet folgende Zuordnung:

Spalte	Bedeutung
Anforderungs-ID	Eindeutige Kennung der Anforderung
Lastenheft-Anforderung	Ursprüngliche Benutzer- bzw. Systemanforderung
Pflichtenheft-Umsetzung	Technische Realisierung
Test-ID	Zugeordneter Testfall
Testebene	Komponenten-, Integrations-, Simulations- oder Systemtest
Status	Bewertung der Umsetzung

36 C.3 Navigation und Lokalisierung

Lastenheft-ID	Lastenheft-Anforderung	Pflichtenheft-Umsetzung	Test-ID	Testebene	Status
NAV-001	Der Bot muss Hindernisse erkennen	Integration einer Abstandssensorik mit Hinderniserkennung	CT-NAV-002, IT-SEN-NAV-001, ST-SIM-002, SYS-003	Komponenten / Integration / Simulation / System	geprüft

ID	Lastenheft- Anforderung	Pflichtenheft- Umsetzung	Test-ID	Testebene	Status
NAV-002	Der Bot muss Hindernissen ausweichen	Dynamische Anpassung der Navigationsroute	CT-NAV-002, IT-SEN-NAV-001, ST-SIM-002	Komponenten / Integration / Simulation	geprüft
NAV-003	Der Bot muss Tischende erkennen	Implementierung einer Tischkantenerkennung	ST-SIM-003, SYS-004	Simulation / System	geprüft
NAV-004	Der Bot muss auf dem Tisch bleiben	Sicherheitslogik verhindert Verlassen der Tischfläche	SYS-004	System	geprüft
NAV-005	Der Bot muss Fahrroute dynamisch anpassen	Navigationsmodul mit Routenneuberechnung	CT-NAV-001, ST-SIM-002	Komponenten / Simulation	geprüft
NAV-006	Der Bot muss Position bestimmen können	Lokalisierungskomponente verarbeitet Positionsdaten	CT-LOC-001, IT-LOC-NAV-001	Komponenten / Integration	geprüft
NAV-007	Der Bot muss Orientierung bestimmen können	Speicherung und Verarbeitung der Roboterorientierung	CT-LOC-001, IT-LOC-NAV-001	Komponenten / Integration	geprüft
NAV-008	Der Bot muss Zielpunkt erreichen	Navigation und Bewegungssteuerung führen Zielfahrt aus	CT-NAV-001, SYS-002	Komponenten / System	geprüft
NAV-009	Der Bot muss sich kontrolliert bewegen	Bewegungssteuerung mit Geschwindigkeitsbegrenzung	CT-MOT-001, IT-NAV-MOT-001	Komponenten / Integration	geprüft

37 C.4 Anforderungen Getränkeuntersetzter

ID	Lastenheft- Anforderung	Pflichtenheft- Umsetzung	Test-ID	Testebene	Status
CST-001	Der Bot muss einen Untersetzter aufnehmen können	Mechanismus zur Aufnahme und Erkennung eines Untersetzers	CT-CST-001, ST-SIM-007, SYS-005	Komponenten / Simulation / System	geprüft

ID	Lastenheft- Anforderung	Pflichtenheft- Umsetzung	Test-ID	Testebene	Status
CST-002	Der Bot muss einen Untersetzer transportieren können	Interne Zustandsverwaltung für aufgenommenen Untersetzer	ST-SIM-007, SYS-010	Simulation / System	geprüft
CST-003	Der Bot muss einen Untersetzer präzise platzieren	Platzierungsmechanismus mit Positionskontrolle	SYS-006	System	geprüft
CST-004	Der Bot muss einen Untersetzer wieder aufnehmen können	Aufnahmeprozess mit Zustandserkennung	CT-CST-001	Komponenten	geprüft
CST-005	Der Bot muss erfolgreiche Aufnahme erkennen	Sensorbasierte Zustandsprüfung	CT-CST-001, CT-CST-002	Komponenten	geprüft
CST-006	Der Bot muss erfolgreiche Platzierung erkennen	Statusprüfung nach Ablagevorgang	SYS-006	System	geprüft

38 C.5 Sicherheitsanforderungen

ID	Lastenheft- Anforderung	Pflichtenheft- Umsetzung	Test-ID	Testebene	Status
SAF-001	Der Bot muss bei Gefahr sofort anhalten	Sicherheitskomponente mit Stoppsignal	CT-SAF-001, IT-SAF-MOT-001, SYS-007	Komponenten / Integration / System	geprüft
SAF-002	Der Bot muss bei Sensorfehler sicheren Zustand einnehmen	Fehlerüberwachung und Sicherheitszustand	ST-SIM-005	Simulation	geprüft
SAF-003	Der Bot darf Tisch nicht verlassen	Tischkantenerkennung und Bewegungsstopp	SYS-004	System	geprüft
SAF-004	Der Bot darf nicht mit hoher Geschwindigkeit kollidieren	Begrenzung der Bewegungsparameter	CT-MOT-001, IT-NAV-MOT-001	Komponenten / Integration	geprüft
SAF-005	Der Bot muss Kollisionen vermeiden	Hinderniserkennung und Navigation	SYS-003	System	geprüft
SAF-006	Der Bot muss Not-Aus unterstützen	Not-Aus-Schnittstelle und Sicherheitslogik	CT-SAF-001, SYS-007	Komponenten / System	geprüft

39 C.6 Systemüberwachung

ID	Lastenheft- Anforderung	Pflichtenheft- Umsetzung	Test-ID	Testebene	Status
MON-001	Fehler erkennen	Diagnosekomponente überwacht Systemzustände	CT-MON-001, IT-MON-001, SYS-008	Komponenten / Integration / System	geprüft
MON-002	Fehler protokollieren	Fehlerlogging innerhalb der Systemüberwachung	CT-MON-001, IT-MON-001	Komponenten / Integration	geprüft
MON-003	Betriebszustand anzeigen	Statusverwaltung und Anzeige	SYS-001	System	geprüft
MON-004	Erfolg einer Aufgabe erkennen	Zustandsmanagement der Arbeitsabläufe	SYS-010	System	geprüft
MON-005	Fehlgeschlagene Aufgaben melden	Fehlerstatus und Meldesystem	CT-CST-002, SYS-008	Komponenten / System	geprüft

40 C.7 Nichtfunktionale Anforderungen

ID	Lastenheft- Anforderung	Pflichtenheft- Umsetzung	Test-ID	Testebene	Status
NFR-001	Bot muss modular aufgebaut sein	Komponentenbasierte Systemarchitektur	IT-HW-SIM-001	Integration	geprüft
NFR-002	Software muss komponentenbasiert entwickelt werden	Trennung der Softwaremodule	CT-HW-001 bis CT-MON-001	Komponentengeprüft	
NFR-003	Softwarearchitektur muss erweiterbar sein	Erweiterbare Schnittstellenstruktur	IT-HW-SIM-001	Integration	geprüft
NFR-004	Software muss simuliert werden können	Hardwareabstraktion und Simulationsmodelle	IT-HW-SIM-001, ST-SIM-001	Integration / Simulation	geprüft
NFR-005	Software muss testbar sein	Automatisierte Testmöglichkeiten	CT-Tests	Komponentengeprüft	
NFR-006	Kernfunktionen müssen automatisiert testbar sein	Testframework und automatisierte Testfälle	CT-Tests	Komponentengeprüft	

ID	Lastenheft-Anforderung	Pflichtenheft-Umsetzung	Test-ID	Testebene	Status
NFR-007	Sensoren und Aktoren müssen austauschbar sein	Hardwareabstraktionsschicht	CT-HW-001, IT-HW-SIM-001	Komponentengeprüft / Integration	geprüft
NFR-008	Hardwareabstraktion unterstützen	Abstrakte Schnittstellen zwischen Hard- und Software	CT-HW-001	Komponentengeprüft	geprüft

41 C.8 Simulationsanforderungen

ID	Lastenheft-Anforderung	Pflichtenheft-Umsetzung	Test-ID	Testebene	Status
SIM-001	Fahrmanöver müssen reproduzierbar sein	Definierte Simulationsszenarien	ST-SIM-004	Simulation	geprüft
SIM-002	Sensoren müssen modellierbar sein	Virtuelle Sensormodelle	IT-HW-SIM-001	Integration	geprüft
SIM-003	Aktoren müssen modellierbar sein	Virtuelle Aktormodelle	ST-SIM-006	Simulation	geprüft
SIM-004	Hindernisse müssen berücksichtigt werden	Virtuelle Umgebungsmodelle	ST-SIM-002	Simulation	geprüft
SIM-005	Fehlersituationen müssen simulierbar sein	Fehlerinjektion in Simulation	ST-SIM-005, ST-SIM-006	Simulation	geprüft
SIM-006	Simulation unterstützt Testfälle	Testautomatisierung innerhalb Simulation	ST-SIM-007	Simulation	geprüft
SIM-007	Simulationsverhalten entspricht erwartetem Verhalten	Vergleich Simulation und Prototyp	ST-SIM-008	Simulation	geprüft

42 C.9 Anforderungen außerhalb des Projektumfangs

Die folgenden Anforderungen wurden bewusst nicht umgesetzt und sind daher nicht Bestandteil der Testabdeckung:

Bereich	Begründung
Getränketransport (Maturity Level 5)	Außerhalb des Projektfokus
Bestellung und Bezahlung	Erweiterungsstufe nach Projektabschluss
Reinigung	Erweiterungsstufe nach Projektabschluss
Netzwerkcommunication	Nicht Bestandteil der Systemarchitektur
Schwarmverhalten	Nicht Bestandteil des Einzelroboters
EMV-Betrachtungen	Nicht Bestandteil der Prototypentwicklung

43 C.10 Zusammenfassung

Die Traceability-Matrix zeigt die vollständige Verbindung zwischen Anforderungen, technischer Umsetzung und Testfällen.

Durch die eindeutige Zuordnung wird sichergestellt, dass die relevanten Anforderungen des Coasterbot-Projekts überprüft werden können.

Die Matrix ermöglicht:

- Nachweis der Anforderungsabdeckung
- strukturierte Testplanung
- transparente Bewertung des Entwicklungsstands
- nachvollziehbare Verifikation des Prototyps

Damit stellt sie einen wichtigen Bestandteil der technischen Dokumentation des Projekts dar.

43.0.1 Anhang D: Risikoanalyse und Testpriorisierung.

44 Anhang D: Risikoanalyse und Testpriorisierung

44.1 D.1 Zweck der Risikoanalyse

Die Risikoanalyse dient der systematischen Identifikation, Bewertung und Behandlung potenzieller Risiken während der Entwicklung, Integration und Validierung des Coasterbot-Prototyps.

Aufgrund der Kombination aus mobiler Robotik, Sensorik, Aktorik und sicherheitskritischen Funktionen entstehen verschiedene technische Risiken, die den Projekterfolg beeinflussen können.

Ziel der Risikoanalyse ist es:

- kritische Systemrisiken frühzeitig zu erkennen

- geeignete Gegenmaßnahmen zu definieren
- Testaktivitäten entsprechend ihrer Bedeutung zu priorisieren
- die Zuverlässigkeit des Prototyps sicherzustellen

Die Bewertung orientiert sich an den Kriterien:

- Eintrittswahrscheinlichkeit
- Auswirkung
- Risikopriorität
- erforderliche Maßnahmen

45 D.2 Bewertungsmethode

Die Risikobewertung erfolgt anhand einer qualitativen Bewertungsskala.

45.1 Eintrittswahrscheinlichkeit

Wert	Beschreibung
1	Sehr unwahrscheinlich
2	Unwahrscheinlich
3	Möglich
4	Wahrscheinlich
5	Sehr wahrscheinlich

45.2 Auswirkung

Wert	Beschreibung
1	Keine relevante Auswirkung
2	Geringe Einschränkung
3	Funktionale Einschränkung
4	Wesentliche Beeinträchtigung
5	Kritischer Systemausfall oder Sicherheitsrisiko

45.3 Risikopriorität

Die Risikopriorität ergibt sich aus:

$$RPZ = \text{Eintrittswahrscheinlichkeit} \times \text{Auswirkung}$$

Die Bewertung erfolgt nach:

RPZ	Bewertung
1–5	Niedriges Risiko
6–12	Mittleres Risiko
13–25	Hohes Risiko

46 D.3 Technische Risikoanalyse

ID	Risiko	Ursache	Eintritt	Auswirkung	RPZ	Bewertung
R-001	Roboter verlässt Tischfläche	Fehlerhafte Kantenerkennung oder Sensorfehler	2	5	10	Mittel
R-002	Kollision mit Hindernissen	Unzureichende Hinderniserkennung	3	5	15	Hoch
R-003	Ungenauere Positionierung	Fehlerhafte Lokalisierung	3	4	12	Mittel
R-004	Untersetzer kann nicht aufgenommen werden	Mechanischer Fehler	3	3	9	Mittel
R-005	Kommunikationsfehler zwischen Softwaremodulen	Fehlerhafte Schnittstellen	2	4	8	Mittel
R-006	Sensor liefert fehlerhafte Daten	Defekter oder falsch konfigurierter Sensor	3	4	12	Mittel
R-007	Bewegungssteuerung reagiert fehlerhaft	Fehler in Motorsteuerung	2	5	10	Mittel
R-008	Simulation entspricht nicht der Realität	Ungenauere Modellierung	3	3	9	Mittel
R-009	Unzureichende Akkukapazität	Falsche Dimensionierung	2	3	6	Mittel
R-010	Softwarefehler führt zu undefiniertem Zustand	Fehlerhafte Zustandsverwaltung	2	5	10	Mittel

47 D.4 Sicherheitsrisiken

47.1 D.4.1 Verlust der Tischsicherheit

47.1.1 Beschreibung

Ein wesentliches Risiko besteht darin, dass der Roboter aufgrund fehlerhafter Sensorinformationen oder einer falschen Bewegungsplanung die Tischfläche verlässt.

47.1.2 Auswirkungen

- Beschädigung des Prototyps
- Unterbrechung des Betriebs
- Potenzielle Gefährdung in realer Umgebung

47.1.3 Gegenmaßnahmen

- redundante Prüfung kritischer Sensordaten
- Begrenzung der Geschwindigkeit
- Sicherheitsabstand zur Tischkante
- Simulation verschiedener Grenzsituationen

47.1.4 Zugeordnete Tests

- SYS-004: Schutz vor Herunterfallen vom Tisch
- ST-SIM-003: Simulation der Tischkantenerkennung
- SYS-007: Sicherheitsstopp

47.2 D.4.2 Kollision mit Objekten oder Personen

47.2.1 Beschreibung

Der Coasterbot bewegt sich in einer Umgebung mit wechselnden Hindernissen und potenziellen Kontaktpunkten.

47.2.2 Auswirkungen

- Beschädigung von Gegenständen
- Fehlfunktion des Arbeitsablaufs
- Sicherheitsrisiko

47.2.3 Gegenmaßnahmen

- Hinderniserkennung
- Geschwindigkeitsbegrenzung
- Kollisionsvermeidung
- Sicherheitsabschaltung

47.2.4 Zugeordnete Tests

- SYS-003: Hinderniserkennung und Umfahrung
- CT-NAV-002: Hinderniserkennung
- SYS-007: Not-Aus

48 D.5 Softwarerisiken

48.1 D.5.1 Fehlerhafte Systemzustände

48.1.1 Beschreibung

Komplexe Robotersysteme besitzen zahlreiche Betriebszustände. Fehlerhafte Übergänge können zu unerwartetem Verhalten führen.

48.1.2 Auswirkungen

- Blockierte Arbeitsabläufe
- Falsche Bewegungsbefehle
- Verlust der Systemzuverlässigkeit

48.1.3 Gegenmaßnahmen

- Zustandsautomat für Betriebsabläufe
- definierte Fehlerzustände
- automatisierte Softwaretests
- Fehlerprotokollierung

48.1.4 Zugeordnete Tests

- CT-MON-001
- IT-MON-001
- SYS-008

48.2 D.5.2 Fehlende Hardwareabstraktion

48.2.1 Beschreibung

Eine unzureichende Trennung zwischen Hard- und Software kann spätere Erweiterungen erschweren.

48.2.2 Auswirkungen

- Erhöhter Entwicklungsaufwand
- Schlechtere Testbarkeit
- Eingeschränkte Austauschbarkeit von Komponenten

48.2.3 Gegenmaßnahmen

- modulare Architektur
- definierte Schnittstellen
- simulierte Hardwarekomponenten

48.2.4 Zugeordnete Tests

- CT-HW-001
- IT-HW-SIM-001

49 D.6 Projektrisiken

ID	Risiko	Ursache	Auswirkung	Gegenmaßnahme
PR-001	Verzögerung im Hardwareaufbau	Lieferprobleme oder Fertigungsfehler	Reduzierte Testzeit	Frühzeitige Beschaffung und Planung
PR-002	Umfangsüberschreitung	Erweiterung um nicht geplante Funktionen	Projektverzögerung	Klare Abgrenzung des Projektumfangs
PR-003	Fehlende Testdaten	Unzureichende Vorbereitung	Eingeschränkte Validierung	Frühzeitige Definition der Testszenarien
PR-004	Integration funktioniert nicht rechtzeitig	Unterschiedliche Entwicklungsstände	Verzögerung der Systemtests	Regelmäßige Integrationstests

50 D.7 Testpriorisierung

Die Testpriorisierung erfolgt anhand der Kritikalität der Anforderungen.

Dabei werden folgende Kriterien berücksichtigt:

- Sicherheitsrelevanz

- Bedeutung für die Kernfunktion
- technische Abhängigkeiten
- Auswirkung bei Fehlern

51 D.8 Prioritätsstufen der Tests

51.1 Priorität P1 – Kritische Tests

Diese Tests besitzen höchste Priorität und müssen vor allen weiteren Tests erfolgreich abgeschlossen werden.

Dazu gehören:

Test-ID	Test
SYS-001	Systemstart
SYS-002	Grundlegende Navigation
SYS-003	Hindernisvermeidung
SYS-004	Tischkantenerkennung
SYS-007	Not-Aus
SYS-008	Fehlerbehandlung

Begründung:

Diese Funktionen stellen die grundlegende Sicherheit und Betriebsfähigkeit des Roboters sicher.

51.2 Priorität P2 – Kernfunktionstests

Diese Tests überprüfen die eigentliche Zielanwendung des Coasterbots.

Dazu gehören:

Test-ID	Test
SYS-005	Untersetzeraufnahme
SYS-006	Untersetzerplatzierung
SYS-010	Vollständiger Arbeitsablauf
ST-SIM-007	Simulation des Untersetzerprozesses

Begründung:

Diese Funktionen bestimmen den praktischen Nutzen des Prototyps.

51.3 Priorität P3 – Erweiterungstests

Diese Tests betreffen optionale oder zukünftige Funktionen.

Beispiele:

- Getränketransport
- Bestellung
- Bezahlung
- Reinigung
- Mensch-Roboter-Interaktion

Diese Funktionen sind nicht Bestandteil des aktuellen Projektumfangs.

52 D.9 Maßnahmen zur Risikoreduzierung

Zur Minimierung technischer Risiken werden folgende Maßnahmen eingesetzt:

52.1 Entwicklung

- modulare Softwarearchitektur
- klare Schnittstellendefinition
- kontinuierliche Integration
- frühzeitige Simulation

52.2 Test

- automatisierte Komponententests
- reproduzierbare Simulationstests
- schrittweise Systemintegration
- Fehlerprotokollierung

52.3 Sicherheit

- Begrenzung kritischer Bewegungen
- definierte sichere Zustände
- Not-Aus-Funktion
- Überwachung wichtiger Systemparameter

53 D.10 Zusammenfassung

Die Risikoanalyse zeigt die wesentlichen technischen und organisatorischen Herausforderungen bei der Entwicklung des Coasterbot-Prototyps.

Besonders relevant sind Risiken in den Bereichen:

- autonome Navigation
- Sicherheit
- Sensorverarbeitung
- Softwareintegration
- Systemzuverlässigkeit

Durch eine strukturierte Testpriorisierung werden kritische Funktionen zuerst überprüft und mögliche Fehler frühzeitig erkannt.

Die Kombination aus Risikoanalyse und Teststrategie unterstützt eine kontrollierte Entwicklung des Prototyps und erhöht die Wahrscheinlichkeit, die definierten Projektziele erfolgreich zu erreichen.